Mini project report on

Campus Bootstrap Matchmaking & Student Statistics DB

Submitted in partial fulfilment of the requirements for the award of degree of

Bachelor of Technology

in

Computer Science & Engineering

UE24CS351A – DBMS Project

Submitted by:

Selvaganesh Arunmozhi	**PES2UG24CS451**
Shantanu Milind Tandel	**PES2UG24CS456**

under the guidance of

Dr. Shilpa S

Assistant Professor

PES University

AUG - DEC 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

FACULTY OF ENGINEERING

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)

Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)
Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

CERTIFICATE

This is to certify that the mini project entitled

Campus Bootstrap Matchmaking & Student Statistics DB

is a bonafide work carried out by

Selvaganesh Arunmozhi	**PES2UG24CS451**
Shantanu Milind Tandel	**PES2UG24CS456**

In partial fulfilment for the completion of fifth semester DBMS Project (UE24CS351A) in the Program of Study - Bachelor of Technology in Computer Science and Engineering under rules and regulations of PES University, Bengaluru during the period AUG. 2026 – DEC. 2026. It is certified that all corrections / suggestions indicated for internal assessment have been incorporated in the report. The project has been approved as it satisfies the 5th semester academic requirements in respect of project work.

Signature
Dr. Shilpa S
Assistant Professor

DECLARATION

We hereby declare that the DBMS Project entitled **Campus Bootstrap Matchmaking & Student Statistics DB** has been carried out by us under the guidance of **Dr. Shilpa S, Assistant Professor** and submitted in partial fulfilment of the course requirements for the award of degree of **Bachelor of Technology** in **Computer Science and Engineering** of **PES University, Bengaluru** during the academic semester AUG – DEC 2026.

Selvaganesh Arunmozhi **PES2UG24CS451**

Shantanu Milind Tandel **PES2UG24CS456**

ACKNOWLEDGEMENT

We would like to express our gratitude to Dr. Shilpa S, Department of Computer Science and Engineering, PES University, for her continuous guidance, assistance, and encouragement throughout the development of this UE24CS351A - DBMS Project.

We take this opportunity to thank Dr. Sandesh B J, C, Professor, ChairPerson, Department of Computer Science and Engineering, PES University, for all the knowledge and support we have received from the department.

We are deeply grateful to Prof. Jawahar Doreswamy, Chancellor – PES University, Dr. Suryaprasad J, Vice-Chancellor, PES University for providing us with various opportunities and enlightenment every step of the way. Finally, this DBMS Project could not have been completed without the continual support and encouragement we have received from our families and friends.

ABSTRACT

New students on campus join through a five-day bootstrap programme. They are split into five colour groups, and each group spends one day in each of five hands-on activities run by a faculty mentor. Managed on spreadsheets, the programme suffers from double-booked rooms, late arrivals judged by hand, attendance edits that leave no trace, no live view of students at risk of missing the certificate, and students who only get to know their own group.

This project, **Campus Bootstrap Matchmaking & Student Statistics DB**, is a database-backed web application built on two database paradigms. **MySQL 8** stores the master data, the 5 × 5 rotation, attendance and feedback in a normalised (3NF/BCNF) schema of nine tables. It enforces the programme's rules inside the database through 11 CHECK constraints, 10 foreign keys, 12 triggers, 11 stored procedures, 5 stored functions and 8 views. **Neo4j** models students, activities, mentors, colour groups and branches as a graph, and uses Cypher traversals to recommend teammates, follow-up activities and mentors, and to find how two students are connected.

A **Next.js** application written in TypeScript gives admins, mentors and students their own dashboards, so that every database feature is reached through a real workflow. Enrolment runs as a single transaction, late check-ins are classified by a trigger, every attendance change is audited, feedback is searched with FULLTEXT indexes, and a 5-star rating stored in MySQL becomes an interest edge in Neo4j that changes the student's recommendations. The system is verified by 69 automated unit and database-integration tests, together with end-to-end HTTP tests.

Keywords: MySQL, Neo4j, triggers, stored procedures, transactions, graph database, recommendation, Next.js

TABLE OF CONTENTS

Chapter No.	Title	Page No.
1.	INTRODUCTION	10
2.	PROBLEM DEFINITION	11
3.	ER MODEL	14
4.	ER TO RELATIONAL MAPPING	16
5.	DDL STATEMENTS	19
6.	DML STATEMENTS	28
7.	QUERIES (SIMPLE QUERY AND UPDATE AND DELETE OPERATION, CORRELATED QUERY AND NESTED QUERY)	31
8.	STORED PROCEDURE, FUNCTIONS AND TRIGGERS	47
9.	FRONT END DEVELOPMENT	59
REFERENCES/BIBLIOGRAPHY		79

LIST OF TABLES

Table No.	Title	Page No.
1.1	Choice of database for each kind of data	**10**
2.1	Functional requirements	**12**
3.1	Entities and their attributes	**14**
3.2	Relationships and cardinality ratios	**14**
5.1	Tables and their constraints	**19**
8.1	Stored functions	**47**
8.2	Stored procedures	**47**
8.3	Triggers	**54**
9.1	Technology stack	**59**
9.2	Graph relationships	**69**

LIST OF FIGURES

Figure No.	Title	Page No.
3.1	ER diagram of the Campus Bootstrap database	**15**
4.1	Relational schema of bootstrap_db	**17**
5.1–5.12	DDL statements and output	**20**
6.1–6.6	INSERT statements	**28**
6.7	UPDATE: check-in and check-out times recorded	**29**
6.8	UPDATE: documented absences marked as excused	**30**
6.9	INSERT: feedback from students who attended	**30**
6.10–6.11	Verification of the loaded rows	**30**
7.1–7.4	SELECT, JOIN, GROUP BY, HAVING and ORDER BY queries	**31**
7.5–7.8	Aggregate function queries	**33**
7.9–7.12	UPDATE operations	**34**
7.13–7.16	DELETE operations	**35**
7.17–7.18	Correlated subqueries	**36**
7.19–7.22	Nested subqueries	**38**
7.23–7.27	Creating the views	**39**
7.28–7.32	Querying the views	**41**
7.33–7.35	Transactions	**43**
7.36–7.40	Full-text search	**45**
8.1–8.4	Stored functions	**48**
8.5–8.12	Stored procedures	**49**
8.13–8.20	Calling the functions and procedures	**53**
8.21–8.26	Creating the triggers	**55**

8.27–8.31	Triggers firing on invalid operations	**58**
9.1	Login page	**60**
9.2	Admin: programme overview built from views, functions and analytical queries	**61**
9.3	Admin: student management (enrol, edit, transfer, delete)	**62**
9.4	Admin: rotation grid	**63**
9.5	Admin: attendance risk report and audit log	**63**
9.6	Admin: activities	**64**
9.7	Admin: mentors	**64**
9.8	Admin: feedback review with full-text search	**65**
9.9	Admin: Neo4j graph status and sync	**66**
9.10	Mentor: activities and attendance roster	**67**
9.11	Student: timetable, attendance and rating	**67**
9.12	Student: matchmaking recommendations from Neo4j	**68**
9.13	Node count by label	**70**
9.14	Relationship count by type	**70**
9.15	Students and their colour groups (BELONGS_TO)	**71**
9.16	Students and their branches (STUDIES)	**72**
9.17	Participation of every student in the five activities (PARTICIPATED_IN)	**73**
9.18	Student interests (INTERESTED_IN)	**74**
9.19	Peer connections between students (CONNECTED_TO)	**75**
9.20	Students with their group, branch and interests	**76**
9.21	A sample of the whole graph	**77**

1. INTRODUCTION

Every year, new students begin their time on campus with a five-day **bootstrap programme**. They are divided into five colour groups (Red, Blue, Green, Purple and Yellow), and each group spends one day in each of five hands-on activities: Automobiles and Robotics, Prompt Engineering, Design Thinking, Biotechnology and Electrical Circuits. A faculty mentor runs each activity. The groups rotate so that, on any day, every group is in a different activity, and over the five days every group visits every activity exactly once. This arrangement is a 5 × 5 Latin square.

The programme has two goals: introduce students to many disciplines, and help them find people from other branches to build things with. Coordinators need accurate attendance and feedback while the week is running, and students need help meeting peers beyond their own group.

Campus Bootstrap Matchmaking & Student Statistics DB supports both goals. It is a web application backed by two databases, each used for the kind of data it handles best, as shown in Table 1.1. MySQL is the source of truth for all records. Neo4j holds a graph derived from MySQL plus two kinds of data that exist only as relationships: peer connections and declared interests.

Table 1.1: Choice of database for each kind of data

Data	Database	Reason
Students, mentors, groups, activities, accounts	MySQL	Fixed structure; uniqueness (USN, email) and referential integrity matter more than flexibility
Rotation, attendance, feedback	MySQL	Business rules must hold on every write: one group per activity per day, capacity, the 15-minute late rule, only attendees may rate, an audit trail
Statistics and reports	MySQL	Aggregation over structured rows: GROUP BY, HAVING, window functions, views
Interests, peer connections, recommendations	Neo4j	Questions about paths (friends of friends, shared interests, how two students are connected) are short pattern matches in Cypher but repeated self-joins or recursive queries in SQL
Peer connections and declared interests	Neo4j only	No report or rule depends on them, and they are naturally edges, so no junction table is needed

The application has three roles. **Admins** manage students, mentors, activities, groups and the rotation, and see programme-wide statistics. **Mentors** record attendance for their own activities and review feedback. **Students** see their timetable and attendance, rate the sessions they attended, declare interests, and receive teammate, activity and mentor recommendations.

2. PROBLEM DEFINITION

2.1 Problem Statement

Running the bootstrap programme on spreadsheets causes three problems:

1. **Scheduling and attendance are error-prone.** A room can be double-booked, a group can be sent to the wrong activity, late arrivals are judged by hand, and a record changed after the fact leaves no trace.
2. **Nobody can see how the programme is going while it runs.** Coordinators cannot tell which groups or activities are losing students, who is at risk of missing the certificate, or what students are saying about each session.
3. **Students leave without knowing each other.** The programme exists to help students from different branches find teammates, but the only people a student meets are those in their own colour group.

The aim is a system in which the database itself enforces the programme's rules, produces live statistics, and recommends connections between students.

2.2 Objectives

1. Model the programme in a normalised relational schema in which the rules are enforced by the database: the rotation, capacity, attendance timing, who may give feedback, and auditability.
2. Use advanced SQL where it solves a real problem: constraints, indexes (composite, covering and full-text), views, stored functions and procedures, triggers, cursors, transactions with savepoints and row locks, and window functions.
3. Model the relationships between students, activities and mentors as a graph in Neo4j, and use graph traversal for matchmaking.
4. Provide a web interface with three roles, so that every database feature is reachable through a real workflow.
5. Keep the application secure: hashed passwords, signed sessions, role checks on every endpoint, parameterised queries, and secrets kept in environment variables.
6. Verify the system with automated tests at the unit, database and HTTP levels.

2.3 Scope

In scope: master data (groups, mentors, activities, students, accounts); the 5 × 5 rotation with rooms and session times; attendance with check-in, check-out, late detection, excusals, absences and an append-only audit log; feedback with ratings, comments and full-text search; statistics by group, activity, branch and student, with risk levels and rankings; the interest and connection graph with recommendations; and a three-role web application with authentication.

Out of scope: self-registration by students, more than five groups or activities, email or push notifications, certificate generation, and multiple campuses or programmes.

2.4 Functional Requirements

Table 2.1: Functional requirements

ID	Requirement	Role	Implemented by
FR-1	Sign in with email and password; sign out; change password	All	AppUser, bcrypt
FR-2	Each role is confined to its own dashboard and data	All	Role checks on every endpoint
FR-3	Enrol a student with five participation rows and a login, atomically	Admin	sp_register_student
FR-4	Edit, transfer and delete students; deletion refused once attendance exists	Admin	sp_transfer_student, trg_student_protect
FR-5	Search and filter students by name, USN, group, branch, semester and risk	Admin	vw_student_attendance
FR-6	Manage mentors and assign them to activities	Admin	Transaction; Activity.m_id ON DELETE SET NULL
FR-7	View and edit the rotation; regenerate it before the programme starts	Admin	Schedule, room-clash triggers, sp_generate_rotation
FR-8	Check students in and out, set check-in times, excuse, mark absent	Mentor, Admin	sp_check_in, sp_check_out, trg_part_set_status, sp_mark_absentees
FR-9	Log every attendance status change with the person who made it	System	trg_part_audit
FR-10	Students see their timetable, attendance %, risk and rank in group	Student	vw_student_schedule, fn_attendance_pct, window functions
FR-11	Students rate sessions they attended (one editable rating per activity)	Student	sp_submit_feedback, trg_feedback_validate
FR-12	Staff review feedback with full-text search and rating distributions	Mentor, Admin	FULLTEXT index ft_feedback_comments
FR-13	Programme dashboard: totals, attendance by group, activity and branch, risk	Admin	Views and analytical queries

ID	Requirement	Role	Implemented by
FR-14	Students declare interests and connect with peers	Student	Neo4j INTERESTED_IN, CONNECTED_TO
FR-15	Recommend teammates, activities and mentors, with the reasons for each	Student	Cypher queries
FR-16	Keep Neo4j in step with MySQL	Admin, System	Full sync and per-student refresh

3. ER MODEL

The ER model has six entity types. Student, Bootstrap group, Mentor and Activity describe the people and the programme. Schedule describes one slot of the rotation (a day, a room and a time for one group and one activity), and Feedback holds a student's rating of an activity. Table 3.1 lists their attributes; key attributes are underlined in Fig 3.1.

Table 3.1: Entities and their attributes

Entity	Key	Other attributes
Student	s_id	s_usn, s_name, email, branch
Bootstrap group	g_id	g_name
Mentor	m_id	m_name, dept, designation, email
Activity	a_id	a_name, capacity, description
Schedule	schedule_id	day, room, start_time, end_time
Feedback	f_id	rating, comments, time

Table 3.2: Relationships and cardinality ratios

Relationship	Between	Cardinality	Notes
belongs to	Student – Bootstrap group	N : 1	Every student is in exactly one colour group
scheduled for	Schedule – Bootstrap group	N : 1	Each group has five slots, one per day
assigned	Schedule – Activity	N : 1	Each activity has five slots, one per day
guided by	Activity – Mentor	N : 1	Partial: an activity may have no mentor, and a mentor may run none
participates	Student – Activity	M : N	Attributes p_id, check_in_time, check_out_time, status
gives	Student – Feedback	1 : N	A student gives at most one feedback per activity
about	Feedback – Activity	N : 1	Each feedback is about one activity

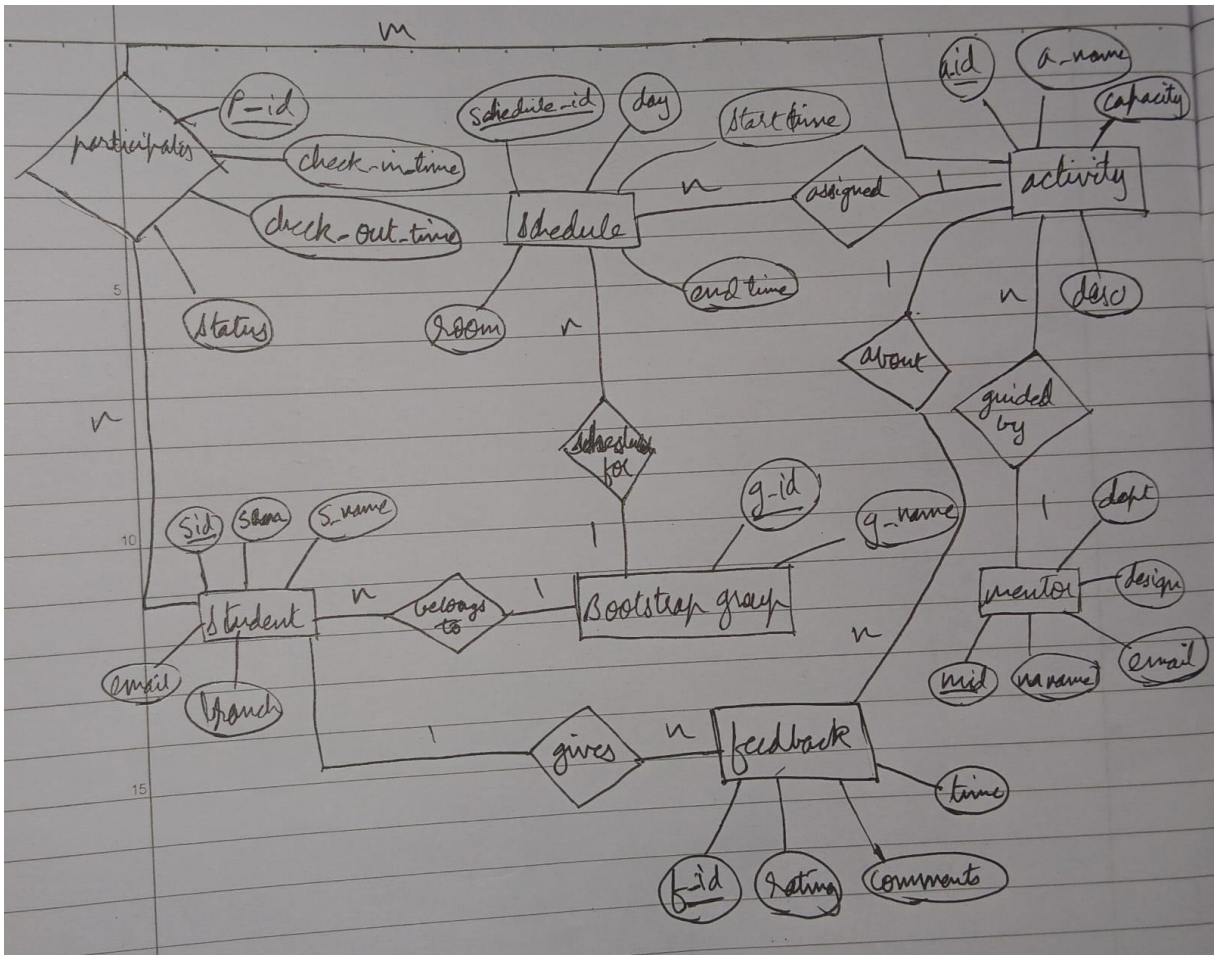


Fig 3.1: ER diagram of the Campus Bootstrap database

Two entities were added during implementation and are not shown in the hand-drawn diagram: **AttendanceLog**, an append-only audit trail of attendance status changes, and **AppUser**, the login account that is linked 1 : 0..1 to a student or a mentor.

4. ER TO RELATIONAL MAPPING

4.1 Steps of the Algorithm for the Chosen Problem

The ER model was converted to relations using the standard seven-step mapping algorithm (Elmasri and Navathe). Each step and its result is listed below.

1. **Regular (strong) entity types.** Each strong entity becomes a relation with its own attributes and key: bootstrap_group(g_id, g_name), Mentor(m_id, ...), Activity(a_id, ...), Student(s_id, ...), Schedule(schedule_id, ...) and Feedback(f_id, ...). Surrogate AUTO_INCREMENT keys are used, and natural keys such as s_usn and email are declared UNIQUE.
2. **Weak entity types.** There are none: every entity has its own key.
3. **Binary 1 : 1 relationships.** Student – AppUser and Mentor – AppUser (1 : 0..1) use the foreign-key approach: AppUser holds s_id and m_id, each declared UNIQUE so that a student or mentor has at most one account.
4. **Binary 1 : N relationships.** The key of the "1" side is added as a foreign key on the "N" side: Student.g_id (belongs to), Schedule.g_id (scheduled for), Schedule.a_id (assigned), Activity.m_id (guided by, nullable because participation is partial), and Feedback.s_id and Feedback.a_id (gives, about).
5. **Binary M : N relationships.** participates becomes its own relation, Participates(p_id, s_id, a_id, check_in_time, check_out_time, status), with foreign keys to Student and Activity and UNIQUE(s_id, a_id), so a student has one row per activity.
6. **Multivalued attributes.** There are none, so no extra relations are needed.
7. **N-ary relationships.** The rotation relates a day, a group and an activity. It is modelled as the Schedule relation with UNIQUE(day, g_id) and UNIQUE(day, a_id), which makes the rotation a Latin square: no group is in two places on one day, and no activity hosts two groups on one day.

Finally, two relations were added for the application. attendance_log stores the history of status changes and has no foreign keys on purpose, so that history survives deletes. AppUser stores login accounts. Derived values such as attendance percentage, risk level, average rating and minutes attended are not stored, which keeps every relation in 3NF/BCNF.

4.2 Complete Diagram of Relational Mapping

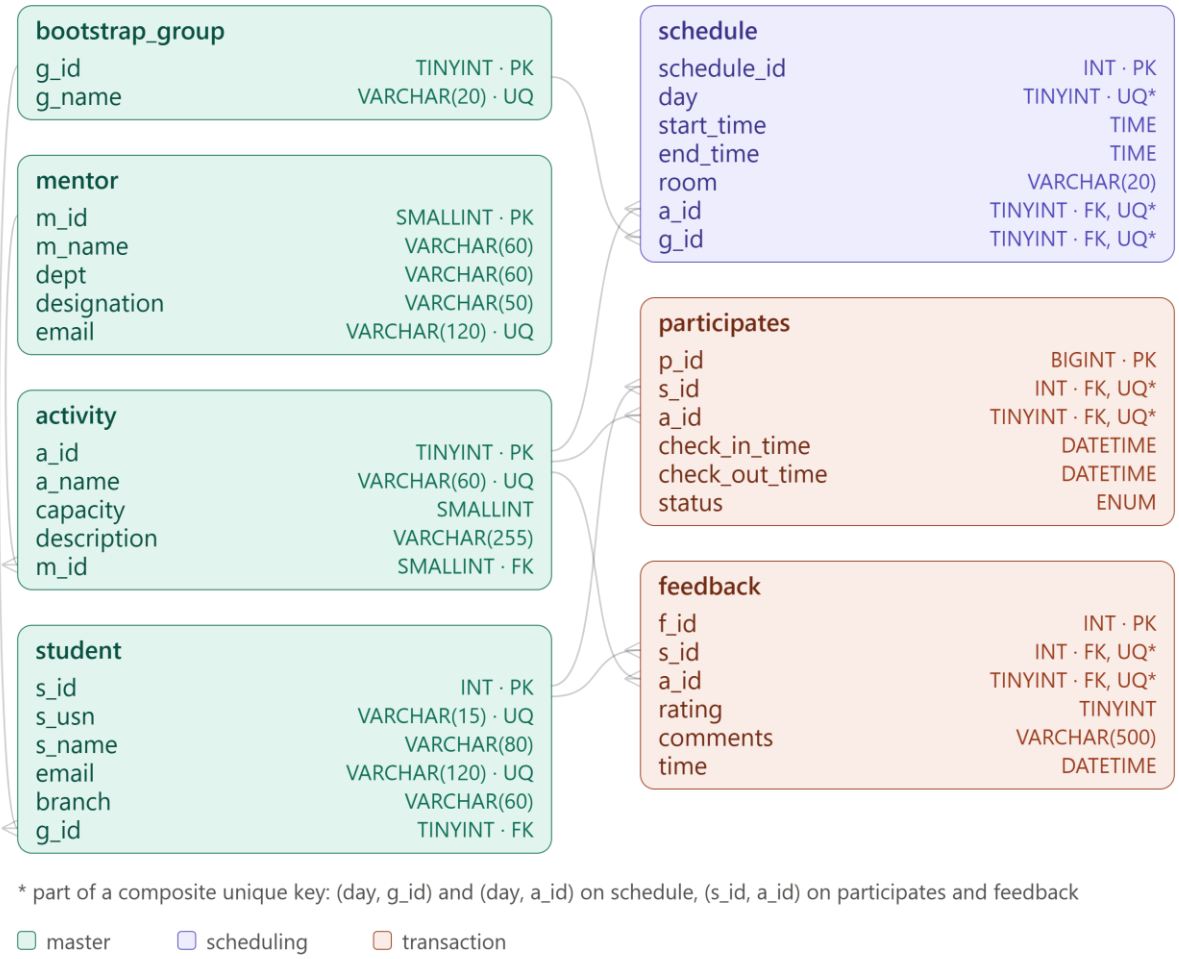


Fig 4.1: Relational schema of bootstrap_db

The resulting relational schema is listed below; primary keys are given first and foreign keys are marked with →.

- **bootstrap_group**(g_id, g_name)
- **Mentor**(m_id, m_name, dept, designation, email, phone)
- **Activity**(a_id, a_name, capacity, description, m_id → Mentor)
- **Student**(s_id, s_usn, s_name, email, branch, semester, g_id → bootstrap_group)
- **Schedule**(schedule_id, day, start_time, end_time, room, a_id → Activity, g_id → bootstrap_group)
- **Participates**(p_id, s_id → Student, a_id → Activity, check_in_time, check_out_time, status)
- **Feedback**(f_id, s_id → Student, a_id → Activity, rating, comments, time)

- **attendance_log**(log_id, p_id, s_id, a_id, old_status, new_status, changed_by, changed_at)
- **AppUser**(user_id, username, password_hash, role, s_id → Student, m_id → Mentor, is_active, last_login, created_at)

5. DDL STATEMENTS

The schema is created by `Campus_Bootstrap_Mini_Project.sql` and extended by `sql/02_app_extensions.sql`. Together they create 9 tables, 8 views, 5 stored functions, 11 stored procedures and 12 triggers. Every table has a primary key, and the foreign-key actions were chosen deliberately: CASCADE for dependent rows, RESTRICT for groups and activities that still have data, and SET NULL for an activity's mentor. Table 5.1 summarises the constraints.

Table 5.1: Tables and their constraints

Table	Primary key	Foreign keys (ON DELETE)	UNIQUE and CHECK constraints
bootstrap_group	g_id	–	UNIQUE(g_name)
Mentor	m_id	–	UNIQUE(email); CHECK phone length ≥ 10
Activity	a_id	m_id → Mentor (SET NULL)	UNIQUE(a_name); CHECK 0 < capacity ≤ 500
Student	s_id	g_id → bootstrap_group (RESTRICT)	UNIQUE(s_usn), UNIQUE(email); CHECK semester 1–8
Schedule	schedule_id	g_id → bootstrap_group (RESTRICT) a_id → Activity (RESTRICT)	UNIQUE(day, g_id), UNIQUE(day, a_id); CHECK day 1–5, end_time > start_time
Participates	p_id	s_id → Student (CASCADE) a_id → Activity (RESTRICT)	UNIQUE(s_id, a_id); CHECK check-out ≥ check-in; an absent row has no times
Feedback	f_id	s_id → Student (CASCADE) a_id → Activity (RESTRICT)	UNIQUE(s_id, a_id); CHECK rating 1–5
attendance_log	log_id	None, by design	–
AppUser	user_id	s_id → Student (CASCADE) m_id → Mentor (CASCADE)	UNIQUE(username), UNIQUE(s_id), UNIQUE(m_id); CHECK bcrypt-format hash, email-format username

All foreign keys use ON UPDATE CASCADE. The DDL also uses ALTER TABLE to add the semester and phone columns and extra constraints, and demonstrates CREATE, ALTER (ADD, RENAME and DROP COLUMN) and DROP on a scratch table. Indexes created for the workload include the composite index `ix_student_group_branch (g_id, branch)`, the covering index `ix_part_activity_status (a_id, status)`, `ix_schedule_day_room (day, room)` for the room-clash triggers, and the FULLTEXT indexes `ft_feedback_comments` and `ft_activity_text`.

Statements with screenshots of table creation

```
mysql> CREATE DATABASE bootstrap_db
    ->     DEFAULT CHARACTER SET utf8mb4
    ->     DEFAULT COLLATE utf8mb4_0900_ai_ci;
Query OK, 1 row affected (0.03 sec)

mysql> Use bootstrap_db;
Database changed
mysql> CREATE TABLE bootstrap_group (
    ->     g_id    TINYINT UNSIGNED NOT NULL AUTO_INCREMENT,
    ->     g_name  VARCHAR(20)      NOT NULL,
    ->
    ->     CONSTRAINT pk_bootstrap_group PRIMARY KEY (g_id),
    ->     CONSTRAINT uq_group_name      UNIQUE (g_name)
    -> );
Query OK, 0 rows affected (0.07 sec)

mysql> CREATE TABLE mentor (
    ->     m_id         SMALLINT UNSIGNED NOT NULL AUTO_INCREMENT,
    ->     m_name       VARCHAR(60)       NOT NULL,
    ->     dept         VARCHAR(60)       NULL,
    ->     designation  VARCHAR(50)       NULL,
    ->     email        VARCHAR(120)      NOT NULL,
    ->
    ->     CONSTRAINT pk_mentor       PRIMARY KEY (m_id),
    ->     CONSTRAINT uq_mentor_email UNIQUE (email)
    -> );
Query OK, 0 rows affected (0.05 sec)
```

Fig 5.1: DDL statements and output (1 of 12)

```
mysql> CREATE TABLE activity (
    ->     a_id         TINYINT UNSIGNED  NOT NULL AUTO_INCREMENT,
    ->     a_name       VARCHAR(60)       NOT NULL,
    ->     capacity     SMALLINT UNSIGNED NOT NULL,
    ->     description  VARCHAR(255)      NULL,
    ->     m_id         SMALLINT UNSIGNED NULL,
    ->
    ->     CONSTRAINT pk_activity      PRIMARY KEY (a_id),
    ->     CONSTRAINT uq_activity_name UNIQUE (a_name),
    ->     CONSTRAINT fk_activity_mentor FOREIGN KEY (m_id)
    ->         REFERENCES mentor (m_id)
    ->         ON DELETE SET NULL ON UPDATE CASCADE,
    ->     CONSTRAINT ck_activity_capacity CHECK (capacity > 0),
    ->
    ->     INDEX ix_activity_mentor (m_id)
    -> );
Query OK, 0 rows affected (0.06 sec)

mysql> CREATE TABLE student (
    ->     s_id    INT UNSIGNED     NOT NULL AUTO_INCREMENT,
    ->     s_usn   VARCHAR(15)      NOT NULL,
    ->     s_name  VARCHAR(80)      NOT NULL,
    ->     email   VARCHAR(120)     NOT NULL,
    ->     branch  VARCHAR(60)      NULL,
    ->     g_id    TINYINT UNSIGNED NOT NULL,
    ->
    ->     CONSTRAINT pk_student       PRIMARY KEY (s_id),
    ->     CONSTRAINT uq_student_usn   UNIQUE (s_usn),
    ->     CONSTRAINT uq_student_email UNIQUE (email),
    ->     CONSTRAINT fk_student_group FOREIGN KEY (g_id)
    ->         REFERENCES bootstrap_group (g_id)
    ->         ON DELETE RESTRICT ON UPDATE CASCADE,
    ->
    ->     INDEX ix_student_group (g_id),
    ->     INDEX ix_student_branch (branch)
    -> );
Query OK, 0 rows affected (0.06 sec)
```

Fig 5.2: DDL statements and output (2 of 12)

```
mysql> CREATE TABLE schedule (
    ->     schedule_id  INT UNSIGNED     NOT NULL AUTO_INCREMENT,
    ->     `day`        TINYINT UNSIGNED NOT NULL,
    ->     start_time   TIME             NOT NULL,
    ->     end_time     TIME             NOT NULL,
    ->     room         VARCHAR(20)      NULL,
    ->     a_id         TINYINT UNSIGNED NOT NULL,
    ->     g_id         TINYINT UNSIGNED NOT NULL,
    ->
    ->     CONSTRAINT pk_schedule PRIMARY KEY (schedule_id),
    ->     CONSTRAINT uq_schedule_group_day UNIQUE (`day`, g_id),
    ->     CONSTRAINT uq_schedule_act_day   UNIQUE (`day`, a_id),
    ->     CONSTRAINT fk_schedule_activity FOREIGN KEY (a_id)
    ->         REFERENCES activity (a_id)
    ->         ON DELETE RESTRICT ON UPDATE CASCADE,
    ->     CONSTRAINT fk_schedule_group FOREIGN KEY (g_id)
    ->         REFERENCES bootstrap_group (g_id)
    ->         ON DELETE RESTRICT ON UPDATE CASCADE,
    ->     CONSTRAINT ck_schedule_day   CHECK (`day` BETWEEN 1 AND 5),
    ->     CONSTRAINT ck_schedule_times CHECK (end_time > start_time),
    ->
    ->     INDEX ix_schedule_activity (a_id),
    ->     INDEX ix_schedule_group (g_id)
    -> );
Query OK, 0 rows affected (0.06 sec)
```

Fig 5.3: DDL statements and output (3 of 12)

```
mysql> CREATE TABLE participates (
    ->     p_id            BIGINT UNSIGNED  NOT NULL AUTO_INCREMENT,
    ->     s_id            INT UNSIGNED     NOT NULL,
    ->     a_id            TINYINT UNSIGNED NOT NULL,
    ->     check_in_time   DATETIME         NULL,
    ->     check_out_time  DATETIME         NULL,
    ->     status          ENUM('present','absent','late','excused')
    ->                     NOT NULL DEFAULT 'present',
    ->
    ->     CONSTRAINT pk_participates PRIMARY KEY (p_id),
    ->     CONSTRAINT uq_participates UNIQUE (s_id, a_id),
    ->     CONSTRAINT fk_part_student FOREIGN KEY (s_id)
    ->         REFERENCES student (s_id)
    ->         ON DELETE CASCADE ON UPDATE CASCADE,
    ->     CONSTRAINT fk_part_activity FOREIGN KEY (a_id)
    ->         REFERENCES activity (a_id)
    ->         ON DELETE RESTRICT ON UPDATE CASCADE,
    ->     CONSTRAINT ck_part_times
    ->         CHECK (check_out_time IS NULL
    ->                OR check_in_time IS NULL
    ->                OR check_out_time >= check_in_time),
    ->     CONSTRAINT ck_part_absent
    ->         CHECK (status <> 'absent'
    ->                OR (check_in_time IS NULL AND check_out_time IS NULL)),
    ->
    ->     INDEX ix_part_activity (a_id),
    ->     INDEX ix_part_status (status)
    -> );
Query OK, 0 rows affected (0.06 sec)
```

Fig 5.4: DDL statements and output (4 of 12)

```
mysql> CREATE TABLE feedback (
    ->     f_id      INT UNSIGNED     NOT NULL AUTO_INCREMENT,
    ->     s_id      INT UNSIGNED     NOT NULL,
    ->     a_id      TINYINT UNSIGNED NOT NULL,
    ->     rating    TINYINT UNSIGNED NOT NULL,
    ->     comments  VARCHAR(500)     NULL,
    ->     `time`    DATETIME         NOT NULL DEFAULT CURRENT_TIMESTAMP,
    ->
    ->     CONSTRAINT pk_feedback      PRIMARY KEY (f_id),
    ->     CONSTRAINT uq_feedback_pair UNIQUE (s_id, a_id),
    ->     CONSTRAINT fk_fb_student FOREIGN KEY (s_id)
    ->         REFERENCES student (s_id)
    ->         ON DELETE CASCADE ON UPDATE CASCADE,
    ->     CONSTRAINT fk_fb_activity FOREIGN KEY (a_id)
    ->         REFERENCES activity (a_id)
    ->         ON DELETE RESTRICT ON UPDATE CASCADE,
    ->     CONSTRAINT ck_fb_rating CHECK (rating BETWEEN 1 AND 5),
    ->
    ->     INDEX ix_fb_activity (a_id)
    -> );
Query OK, 0 rows affected (0.07 sec)

mysql> show tables;
+------------------------+
| Tables_in_bootstrap_db |
+------------------------+
| activity               |
| bootstrap_group        |
| feedback               |
| mentor                 |
| participates           |
| schedule               |
| student                |
+------------------------+
7 rows in set (0.01 sec)

mysql>
```

Fig 5.5: DDL statements and output (5 of 12)

```
mysql> desc activity;
+-------------+------------------+------+-----+---------+----------------+
| Field       | Type             | Null | Key | Default | Extra          |
+-------------+------------------+------+-----+---------+----------------+
| a_id        | tinyint unsigned | NO   | PRI | NULL    | auto_increment |
| a_name      | varchar(60)      | NO   | UNI | NULL    |                |
| capacity    | smallint unsigned| NO   |     | NULL    |                |
| description | varchar(255)     | YES  |     | NULL    |                |
| m_id        | smallint unsigned| YES  | MUL | NULL    |                |
+-------------+------------------+------+-----+---------+----------------+
5 rows in set (0.01 sec)
```

Fig 5.6: DDL statements and output (6 of 12)

```
mysql> desc bootstrap_group;
+--------+------------------+------+-----+---------+----------------+
| Field  | Type             | Null | Key | Default | Extra          |
+--------+------------------+------+-----+---------+----------------+
| g_id   | tinyint unsigned | NO   | PRI | NULL    | auto_increment |
| g_name | varchar(20)      | NO   | UNI | NULL    |                |
+--------+------------------+------+-----+---------+----------------+
2 rows in set (0.00 sec)

mysql> desc feedback;
+----------+------------------+------+-----+-------------------+-------------------+
| Field    | Type             | Null | Key | Default           | Extra             |
+----------+------------------+------+-----+-------------------+-------------------+
| f_id     | int unsigned     | NO   | PRI | NULL              | auto_increment    |
| s_id     | int unsigned     | NO   | MUL | NULL              |                   |
| a_id     | tinyint unsigned | NO   | MUL | NULL              |                   |
| rating   | tinyint unsigned | NO   |     | NULL              |                   |
| comments | varchar(500)     | YES  |     | NULL              |                   |
| time     | datetime         | NO   |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+----------+------------------+------+-----+-------------------+-------------------+
6 rows in set (0.00 sec)

mysql> desc mentor;
+-------------+------------------+------+-----+---------+----------------+
| Field       | Type             | Null | Key | Default | Extra          |
+-------------+------------------+------+-----+---------+----------------+
| m_id        | smallint unsigned | NO   | PRI | NULL    | auto_increment |
| m_name      | varchar(60)      | NO   |     | NULL    |                |
| dept        | varchar(60)      | YES  |     | NULL    |                |
| designation | varchar(50)      | YES  |     | NULL    |                |
| email       | varchar(120)     | NO   | UNI | NULL    |                |
+-------------+------------------+------+-----+---------+----------------+
5 rows in set (0.00 sec)

mysql> desc participates;
+----------------+-----------------------------------------+------+-----+---------+----------------+
| Field          | Type                                    | Null | Key | Default | Extra          |
+----------------+-----------------------------------------+------+-----+---------+----------------+
| p_id           | bigint unsigned                         | NO   | PRI | NULL    | auto_increment |
| s_id           | int unsigned                            | NO   | MUL | NULL    |                |
| a_id           | tinyint unsigned                        | NO   | MUL | NULL    |                |
| check_in_time  | datetime                                | YES  |     | NULL    |                |
| check_out_time | datetime                                | YES  |     | NULL    |                |
| status         | enum('present','absent','late','excused') | NO   | MUL | present |                |
+----------------+-----------------------------------------+------+-----+---------+----------------+
6 rows in set (0.00 sec)
```

Fig 5.7: DDL statements and output (7 of 12)

```
mysql> desc schedule;
+-------------+------------------+------+-----+---------+----------------+
| Field       | Type             | Null | Key | Default | Extra          |
+-------------+------------------+------+-----+---------+----------------+
| schedule_id | int unsigned     | NO   | PRI | NULL    | auto_increment |
| day         | tinyint unsigned | NO   | MUL | NULL    |                |
| start_time  | time             | NO   |     | NULL    |                |
| end_time    | time             | NO   |     | NULL    |                |
| room        | varchar(20)      | YES  |     | NULL    |                |
| a_id        | tinyint unsigned | NO   | MUL | NULL    |                |
| g_id        | tinyint unsigned | NO   | MUL | NULL    |                |
+-------------+------------------+------+-----+---------+----------------+
7 rows in set (0.00 sec)

mysql> desc student;
+--------+------------------+------+-----+---------+----------------+
| Field  | Type             | Null | Key | Default | Extra          |
+--------+------------------+------+-----+---------+----------------+
| s_id   | int unsigned     | NO   | PRI | NULL    | auto_increment |
| s_usn  | varchar(15)      | NO   | UNI | NULL    |                |
| s_name | varchar(80)      | NO   |     | NULL    |                |
| email  | varchar(120)     | NO   | UNI | NULL    |                |
| branch | varchar(60)      | YES  | MUL | NULL    |                |
| g_id   | tinyint unsigned | NO   | MUL | NULL    |                |
+--------+------------------+------+-----+---------+----------------+
6 rows in set (0.00 sec)
```

Fig 5.8: DDL statements and output (8 of 12)

```
mysql> CREATE TABLE attendance_log (
    ->     log_id      BIGINT UNSIGNED  NOT NULL AUTO_INCREMENT,
    ->     p_id        BIGINT UNSIGNED  NOT NULL,
    ->     s_id        INT UNSIGNED     NOT NULL,
    ->     a_id        TINYINT UNSIGNED NOT NULL,
    ->     old_status  VARCHAR(10)      NULL,
    ->     new_status  VARCHAR(10)      NULL,
    ->     changed_by  VARCHAR(100)     NULL,
    ->     changed_at  DATETIME         NOT NULL DEFAULT CURRENT_TIMESTAMP,
    ->
    ->     CONSTRAINT pk_attendance_log PRIMARY KEY (log_id),
    ->     INDEX ix_log_student (s_id),
    ->     INDEX ix_log_activity (a_id)
    -> );
Query OK, 0 rows affected (0.02 sec)
```

Fig 5.9: DDL statements and output (9 of 12)

```
mysql> show tables;
+------------------------+
| Tables_in_bootstrap_db |
+------------------------+
| activity               |
| attendance_log         |
| bootstrap_group        |
| feedback               |
| mentor                 |
| participates           |
| schedule               |
| student                |
+------------------------+
8 rows in set (0.00 sec)
```

Fig 5.10: DDL statements and output (10 of 12)

```
mysql> desc attendance_log
    -> ;
+------------+-----------------+------+-----+-------------------+-------------------+
| Field      | Type            | Null | Key | Default           | Extra             |
+------------+-----------------+------+-----+-------------------+-------------------+
| log_id     | bigint unsigned | NO   | PRI | NULL              | auto_increment    |
| p_id       | bigint unsigned | NO   |     | NULL              |                   |
| s_id       | int unsigned    | NO   | MUL | NULL              |                   |
| a_id       | tinyint unsigned| NO   | MUL | NULL              |                   |
| old_status | varchar(10)     | YES  |     | NULL              |                   |
| new_status | varchar(10)     | YES  |     | NULL              |                   |
| changed_by | varchar(100)    | YES  |     | NULL              |                   |
| changed_at | datetime        | NO   |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+------------+-----------------+------+-----+-------------------+-------------------+
8 rows in set (0.00 sec)
```

Fig 5.11: DDL statements and output (11 of 12)

```
mysql> ALTER TABLE mentor
    ->     ADD COLUMN phone VARCHAR(15) NULL AFTER email;
Query OK, 0 rows affected (0.01 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> ALTER TABLE feedback
    ->     MODIFY COLUMN comments VARCHAR(1000) NULL;
Query OK, 0 rows affected (0.01 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> ALTER TABLE activity
    ->     ADD CONSTRAINT ck_activity_capacity_max CHECK (capacity <= 500);
Query OK, 0 rows affected (0.07 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> ALTER TABLE participates
    ->     ADD INDEX ix_part_checkin (check_in_time);
Query OK, 0 rows affected (0.02 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

Fig 5.12: DDL statements and output (12 of 12)

6. DML STATEMENTS

The seed data is loaded in Part 2 of the SQL script. The rotation is not typed in by hand: `sp_generate_rotation` builds the 25 schedule slots of the Latin square. Participation rows are created with a single `INSERT ... SELECT` over `Student CROSS JOIN Activity`, and each row is checked by the `trg_part_validate` trigger. UPDATE statements then set only the check-in and check-out times; the triggers derive each status (present or late) from those times. Feedback is inserted only for students who attended, as `trg_feedback_validate` requires.

After loading, the database holds 5 colour groups, 10 mentors, 5 activities, 51 students, 25 schedule slots, 255 participation rows, 76 feedback rows and 62 login accounts.

Statements with screenshots of the tables with inserted values

```
mysql> INSERT INTO bootstrap_group (g_id, g_name) VALUES
    -> (1, 'Red'), (2, 'Blue'), (3, 'Green'), (4, 'Purple'), (5, 'Yellow');
Query OK, 5 rows affected (0.00 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

Fig 6.1: INSERT statements (1 of 6)

```
mysql> INSERT INTO mentor (m_id, m_name, dept, designation, email, phone) VALUES
    -> (1, 'Anitha Rao',    'Mechanical Engineering',          'Associate Professor', 'anitha.rao@univ.edu',    '9845010001'),
    -> (2, 'Priya Iyer',    'Computer Science and Engineering', 'Assistant Professor', 'priya.iyer@univ.edu',    '9845010002'),
    -> (3, 'Suresh Pai',    'Design and Innovation',           'Professor',           'suresh.pai@univ.edu',    '9845010003'),
    -> (4, 'Sneha Kamath',  'Biotechnology',                   'Associate Professor', 'sneha.kamath@univ.edu',  '9845010004'),
    -> (5, 'Ramesh Naik',   'Electrical and Electronics',      'Assistant Professor', 'ramesh.naik@univ.edu',   '9845010005'),
    -> (6, 'Vikram Shetty', 'Mechanical Engineering',          'Assistant Professor', 'vikram.shetty@univ.edu', '9845010006'),
    -> (7, 'Meera Joshi',   'Computer Science and Engineering', 'Associate Professor', 'meera.joshi@univ.edu',  '9845010007'),
    -> (8, 'Farhan Ahmed',  'Design and Innovation',           'Assistant Professor', 'farhan.ahmed@univ.edu',  '9845010008');
Query OK, 8 rows affected (0.00 sec)
Records: 8  Duplicates: 0  Warnings: 0
```

Fig 6.2: INSERT statements (2 of 6)

```
mysql> INSERT INTO activity (a_id, a_name, capacity, description, m_id) VALUES
    -> (1, 'Automobiles and Robotics', 60, 'Chassis assembly, servo wiring and line following robot build', 1),
    -> (2, 'Prompt Engineering',       60, 'Working with large language models and structured prompting',   2),
    -> (3, 'Design Thinking',          60, 'Empathise, ideate, paper prototyping and rapid user testing',   3),
    -> (4, 'Biotechnology',            60, 'Wet lab safety, micropipetting technique and gel runs',         4),
    -> (5, 'Electrical Circuits',      25, 'Breadboard layout, soldering practice and circuit analysis',    5);
Query OK, 5 rows affected (0.00 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

Fig 6.3: INSERT statements (3 of 6)

```
mysql> CALL sp_generate_rotation('09:00:00', '16:30:00');
Query OK, 1 row affected (0.03 sec)
```

Fig 6.4: INSERT statements (4 of 6)

```
mysql> INSERT INTO student (s_usn, s_name, email, branch, semester, g_id) VALUES
    -> ('1BM26CS001', 'Aarav Sharma',     'aarav.sharma@univ.edu',     'Computer Science', 1, 1),
    -> ('1BM26EC002', 'Bhavana Nayak',    'bhavana.nayak@univ.edu',    'Electronics',      1, 2),
    -> ('1BM26ME003', 'Chirag Patel',     'chirag.patel@univ.edu',     'Mechanical',       1, 3),
    -> ('1BM26BT004', 'Divya Menon',      'divya.menon@univ.edu',      'Biotechnology',    1, 4),
    -> ('1BM26CV005', 'Eshan Kulkarni',   'eshan.kulkarni@univ.edu',   'Civil',            1, 5),
    -> ('1BM26CS006', 'Farhan Siddiqui',  'farhan.siddiqui@univ.edu',  'Computer Science', 1, 1),
    -> ('1BM26EC007', 'Gayathri Kamath',  'gayathri.kamath@univ.edu',  'Electronics',      1, 2),
    -> ('1BM26ME008', 'Harish Raghavan',  'harish.raghavan@univ.edu',  'Mechanical',       1, 3),
    -> ('1BM26BT009', 'Ishita Verma',     'ishita.verma@univ.edu',     'Biotechnology',    1, 4),
    -> ('1BM26CV010', 'Jatin Prakash',    'jatin.prakash@univ.edu',    'Civil',            1, 5),
    -> ('1BM26CS011', 'Kavya Deshpande',  'kavya.deshpande@univ.edu',  'Computer Science', 2, 1),
    -> ('1BM26EC012', 'Lohith Krishnan',  'lohith.krishnan@univ.edu',  'Electronics',      2, 2),
    -> ('1BM26ME013', 'Manav Gupta',      'manav.gupta@univ.edu',      'Mechanical',       2, 3),
    -> ('1BM26BT014', 'Nandini Shetty',   'nandini.shetty@univ.edu',   'Biotechnology',    2, 4),
    -> ('1BM26CV015', 'Omkar Joshi',      'omkar.joshi@univ.edu',      'Civil',            2, 5),
    -> ('1BM26CS016', 'Pallavi Reddy',    'pallavi.reddy@univ.edu',    'Computer Science', 2, 1),
    -> ('1BM26EC017', 'Qasim Ali',        'qasim.ali@univ.edu',        'Electronics',      2, 2),
    -> ('1BM26ME018', 'Rithika Bhat',     'rithika.bhat@univ.edu',     'Mechanical',       2, 3),
    -> ('1BM26BT019', 'Sagar Hegde',      'sagar.hegde@univ.edu',      'Biotechnology',    2, 4),
    -> ('1BM26CV020', 'Tanvi Iyer',       'tanvi.iyer@univ.edu',       'Civil',            2, 5),
    -> ('1BM26CS021', 'Uday Chandran',    'uday.chandran@univ.edu',    'Computer Science', 1, 1),
    -> ('1BM26EC022', 'Varsha Pillai',    'varsha.pillai@univ.edu',    'Electronics',      1, 2),
    -> ('1BM26ME023', 'Wasim Khan',       'wasim.khan@univ.edu',       'Mechanical',       1, 3),
    -> ('1BM26BT024', 'Yamini Rao',       'yamini.rao@univ.edu',       'Biotechnology',    1, 4),
    -> ('1BM26CV025', 'Zoya Fernandes',   'zoya.fernandes@univ.edu',   'Civil',            1, 5),
    -> ('1BM26CS026', 'Aditya Rane',      'aditya.rane@univ.edu',      'Computer Science', 2, 1),
    -> ('1BM26EC027', 'Bhoomika Jain',    'bhoomika.jain@univ.edu',    'Electronics',      2, 2),
    -> ('1BM26ME028', 'Chetan Murthy',    'chetan.murthy@univ.edu',    'Mechanical',       2, 3),
    -> ('1BM26BT029', 'Deepika Shenoy',   'deepika.shenoy@univ.edu',   'Biotechnology',    2, 4),
    -> ('1BM26CV030', 'Eknath Salian',    'eknath.salian@univ.edu',    'Civil',            2, 5),
    -> ('1BM26CS031', 'Fiza Mirza',       'fiza.mirza@univ.edu',       'Computer Science', 1, 1),
    -> ('1BM26EC032', 'Girish Acharya',   'girish.acharya@univ.edu',   'Electronics',      1, 2),
    -> ('1BM26ME033', 'Hemanth Kini',     'hemanth.kini@univ.edu',     'Mechanical',       1, 3),
    -> ('1BM26BT034', 'Indira Varma',     'indira.varma@univ.edu',     'Biotechnology',    1, 4),
    -> ('1BM26CV035', 'Jeevan Poojary',   'jeevan.poojary@univ.edu',   'Civil',            1, 5),
    -> ('1BM26CS036', 'Keerthi Suresh',   'keerthi.suresh@univ.edu',   'Computer Science', 2, 1),
    -> ('1BM26EC037', 'Lalit Bansal',     'lalit.bansal@univ.edu',     'Electronics',      2, 2),
    -> ('1BM26ME038', 'Meghana Rau',      'meghana.rau@univ.edu',      'Mechanical',       2, 3),
    -> ('1BM26BT039', 'Nikhil Dsouza',    'nikhil.dsouza@univ.edu',    'Biotechnology',    2, 4),
    -> ('1BM26CV040', 'Oviya Natarajan',  'oviya.natarajan@univ.edu',  'Civil',            2, 5),
    -> ('1BM26CS041', 'Pranav Hebbar',    'pranav.hebbar@univ.edu',    'Computer Science', 1, 1),
    -> ('1BM26EC042', 'Rakshita Udupa',   'rakshita.udupa@univ.edu',   'Electronics',      1, 2),
    -> ('1BM26ME043', 'Siddharth Menon',  'siddharth.menon@univ.edu',  'Mechanical',       1, 3),
    -> ('1BM26BT044', 'Trisha Baliga',    'trisha.baliga@univ.edu',    'Biotechnology',    1, 4),
    -> ('1BM26CV045', 'Ujwal Kadam',      'ujwal.kadam@univ.edu',      'Civil',            1, 5),
    -> ('1BM26CS046', 'Vaishnavi Prabhu', 'vaishnavi.prabhu@univ.edu', 'Computer Science', 2, 1),
    -> ('1BM26EC047', 'Yash Thakur',      'yash.thakur@univ.edu',      'Electronics',      2, 2),
    -> ('1BM26ME048', 'Zara Qureshi',     'zara.qureshi@univ.edu',     'Mechanical',       2, 3),
    -> ('1BM26BT049', 'Akhil Ramanna',    'akhil.ramanna@univ.edu',    'Biotechnology',    2, 4),
    -> ('1BM26CV050', 'Bindu Shastry',    'bindu.shastry@univ.edu',    'Civil',            2, 5);
Query OK, 50 rows affected (0.01 sec)
Records: 50  Duplicates: 0  Warnings: 0
```

Fig 6.5: INSERT statements (5 of 6)

```
mysql> INSERT INTO participates (s_id, a_id, status)
    -> SELECT s.s_id, a.a_id, 'absent'
    -> FROM   student s
    -> CROSS JOIN activity a;
Query OK, 250 rows affected (0.13 sec)
Records: 250  Duplicates: 0  Warnings: 0
```

Fig 6.6: INSERT statements (6 of 6)

```
mysql> UPDATE  participates p
    -> JOIN    student  s  ON s.s_id  = p.s_id
    -> JOIN    schedule sc ON sc.g_id = s.g_id AND sc.a_id = p.a_id
    -> SET     p.check_in_time = TIMESTAMP(
    ->             DATE_ADD('2026-10-05', INTERVAL sc.`day` - 1 DAY),
    ->             ADDTIME(sc.start_time,
    ->                     SEC_TO_TIME(((p.s_id * 7 + p.a_id * 13) MOD 35) * 60))),
    ->         p.check_out_time = TIMESTAMP(
    ->             DATE_ADD('2026-10-05', INTERVAL sc.`day` - 1 DAY),
    ->             sc.end_time)
    -> WHERE   (p.s_id * 11 + p.a_id * 5) MOD 10 <> 0;
Query OK, 225 rows affected (0.12 sec)
Rows matched: 225  Changed: 225  Warnings: 0
```

Fig 6.7: UPDATE: check-in and check-out times recorded

```
mysql> UPDATE  participates
    -> SET     status = 'excused'
    -> WHERE   check_in_time IS NULL
    ->   AND   (s_id + a_id) MOD 4 = 1;
Query OK, 0 rows affected (0.00 sec)
Rows matched: 0  Changed: 0  Warnings: 0
```

Fig 6.8: UPDATE: documented absences marked as excused

```
mysql> INSERT INTO feedback (s_id, a_id, rating, comments)
    -> SELECT  p.s_id,
    ->         p.a_id,
    ->         1 + ((p.s_id * 5 + p.a_id * 11) MOD 5),
    ->         CASE (p.s_id + p.a_id) MOD 8
    ->             WHEN 0 THEN 'Excellent hands on session, the soldering practice was easily the best part of the week.'
    ->             WHEN 1 THEN 'The mentor explained the theory clearly but we ran short of time near the end.'
    ->             WHEN 2 THEN 'Lab was too crowded, had to share equipment with three other students for most of the day.'
    ->             WHEN 3 THEN 'Loved building the robot chassis, would happily spend a second day on this activity.'
    ->             WHEN 4 THEN 'Good introduction overall but the prototyping tools were outdated and slow.'
    ->             WHEN 5 THEN 'The group exercise on ideation was engaging and very well paced throughout.'
    ->             WHEN 6 THEN 'Felt rushed, the circuit analysis needed more worked examples before the practical.'
    ->             ELSE        'Well organised, clear instructions and the teaching assistants were genuinely helpful.'
    ->         END
    -> FROM    participates p
    -> WHERE   p.status IN ('present', 'late')
    ->   AND   (p.s_id + p.a_id) MOD 3 = 0;
Query OK, 75 rows affected (0.02 sec)
Records: 75  Duplicates: 0  Warnings: 0
```

Fig 6.9: INSERT: feedback from students who attended

```
mysql> -- 9. Verification ----------------------------------------------
mysql> SELECT 'bootstrap_group' AS table_name, COUNT(*) AS rows_loaded FROM bootstrap_group
    -> UNION ALL SELECT 'mentor',         COUNT(*) FROM mentor
    -> UNION ALL SELECT 'activity',       COUNT(*) FROM activity
    -> UNION ALL SELECT 'schedule',       COUNT(*) FROM schedule
    -> UNION ALL SELECT 'student',        COUNT(*) FROM student
    -> UNION ALL SELECT 'participates',   COUNT(*) FROM participates
    -> UNION ALL SELECT 'feedback',       COUNT(*) FROM feedback
    -> UNION ALL SELECT 'attendance_log', COUNT(*) FROM attendance_log;
+-----------------+-------------+
| table_name      | rows_loaded |
+-----------------+-------------+
| bootstrap_group |           5 |
| mentor          |           8 |
| activity        |           5 |
| schedule        |          25 |
| student         |          50 |
| participates    |         250 |
| feedback        |          75 |
| attendance_log  |         225 |
+-----------------+-------------+
8 rows in set (0.01 sec)
```

Fig 6.10: Verification of the loaded rows (1 of 2)

```
mysql> SELECT status, COUNT(*) AS rows_with_status
    -> FROM   participates
    -> GROUP  BY status;
+---------+-------------------+
| status  | rows_with_status  |
+---------+-------------------+
| present |                90 |
| absent  |                25 |
| late    |               135 |
+---------+-------------------+
3 rows in set (0.00 sec)
```

Fig 6.11: Verification of the loaded rows (2 of 2)

7. QUERIES

7.1 Simple Query with GROUP BY, Aggregate

The queries below are in Part 5 of the SQL script. They include an INNER JOIN that prints a group's timetable, a LEFT JOIN that lists every mentor (including those not assigned to an activity), GROUP BY with COUNT, SUM and AVG to compute attendance per activity, HAVING to filter branches and activities below a threshold, and ORDER BY with LIMIT for the ten best attenders. An example is:

```
SELECT a.a_name, COUNT(p.p_id) AS enrolled,
       SUM(p.status = 'present') AS present, SUM(p.status = 'late') AS late_arrivals,
       SUM(p.status = 'absent')  AS absent
FROM   Activity a JOIN Participates p ON p.a_id = a.a_id
GROUP  BY a.a_id, a.a_name;
```

```
mysql> SELECT      s_usn, s_name, branch, semester
    -> FROM        student
    -> WHERE       branch = 'Computer Science'
    -> ORDER BY    s_name
    -> LIMIT       10;
+------------+------------------+------------------+----------+
| s_usn      | s_name           | branch           | semester |
+------------+------------------+------------------+----------+
| 1BM26CS001 | Aarav Sharma     | Computer Science |        1 |
| 1BM26CS026 | Aditya Rane      | Computer Science |        2 |
| 1BM26CS006 | Farhan Siddiqui  | Computer Science |        1 |
| 1BM26CS031 | Fiza Mirza       | Computer Science |        1 |
| 1BM26CS011 | Kavya Deshpande  | Computer Science |        2 |
| 1BM26CS036 | Keerthi Suresh   | Computer Science |        2 |
| 1BM26CS016 | Pallavi Reddy    | Computer Science |        2 |
| 1BM26CS041 | Pranav Hebbar    | Computer Science |        1 |
| 1BM26CS021 | Uday Chandran    | Computer Science |        1 |
| 1BM26CS046 | Vaishnavi Prabhu | Computer Science |        2 |
+------------+------------------+------------------+----------+
10 rows in set (0.00 sec)
```

Fig 7.1: SELECT, JOIN, GROUP BY, HAVING and ORDER BY queries (1 of 4)

```
mysql> SELECT      sc.`day`,
    ->             g.g_name   AS colour_group,
    ->             a.a_name   AS activity,
    ->             sc.room,
    ->             sc.start_time,
    ->             sc.end_time
    -> FROM        schedule        sc
    -> JOIN        bootstrap_group g ON g.g_id = sc.g_id
    -> JOIN        activity        a ON a.a_id = sc.a_id
    -> WHERE       g.g_name = 'Green'
    -> ORDER BY    sc.`day`;
+-----+--------------+--------------------------+---------+------------+----------+
| day | colour_group | activity                 | room    | start_time | end_time |
+-----+--------------+--------------------------+---------+------------+----------+
|   1 | Green        | Design Thinking          | ROOM-03 | 09:00:00   | 16:30:00 |
|   2 | Green        | Biotechnology            | ROOM-04 | 09:00:00   | 16:30:00 |
|   3 | Green        | Electrical Circuits      | ROOM-05 | 09:00:00   | 16:30:00 |
|   4 | Green        | Automobiles and Robotics | ROOM-01 | 09:00:00   | 16:30:00 |
|   5 | Green        | Prompt Engineering       | ROOM-02 | 09:00:00   | 16:30:00 |
+-----+--------------+--------------------------+---------+------------+----------+
5 rows in set (0.00 sec)
```

Fig 7.2: SELECT, JOIN, GROUP BY, HAVING and ORDER BY queries (2 of 4)

```
mysql> SELECT      m.m_name,
    ->             m.dept,
    ->             a.a_name AS assigned_activity
    -> FROM        mentor   m
    -> LEFT JOIN   activity a ON a.m_id = m.m_id
    -> ORDER BY    a.a_name IS NULL, m.m_name;
+---------------+-----------------------------------+--------------------------+
| m_name        | dept                              | assigned_activity        |
+---------------+-----------------------------------+--------------------------+
| Anitha Rao    | Mechanical Engineering            | Automobiles and Robotics |
| Priya Iyer    | Computer Science and Engineering  | Prompt Engineering       |
| Ramesh Naik   | Electrical and Electronics        | Electrical Circuits      |
| Sneha Kamath  | Biotechnology                     | Biotechnology            |
| Suresh Pai    | Design and Innovation             | Design Thinking          |
| Farhan Ahmed  | Design and Innovation             | NULL                     |
| Meera Joshi   | Computer Science and Engineering  | NULL                     |
| Vikram Shetty | Mechanical Engineering            | NULL                     |
+---------------+-----------------------------------+--------------------------+
8 rows in set (0.00 sec)
```

Fig 7.3: SELECT, JOIN, GROUP BY, HAVING and ORDER BY queries (3 of 4)

```
mysql> SELECT      branch,
    ->             COUNT(*) AS students
    -> FROM        student
    -> GROUP BY    branch
    -> HAVING      COUNT(*) > 8
    -> ORDER BY    students DESC;
+------------------+----------+
| branch           | students |
+------------------+----------+
| Computer Science |       10 |
| Electronics      |       10 |
| Mechanical       |       10 |
| Biotechnology    |       10 |
| Civil            |       10 |
+------------------+----------+
5 rows in set (0.00 sec)
```

Fig 7.4: SELECT, JOIN, GROUP BY, HAVING and ORDER BY queries (4 of 4)

```
mysql> SELECT      a.a_name,
    ->             COUNT(p.p_id)                                  AS enrolled,
    ->             SUM(p.status IN ('present','late'))            AS attended,
    ->             ROUND(100 * SUM(p.status IN ('present','late'))
    ->                       / COUNT(p.p_id), 2)                  AS attendance_pct
    -> FROM        activity     a
    -> JOIN        participates p ON p.a_id = a.a_id
    -> GROUP BY    a.a_id, a.a_name
    -> HAVING      attendance_pct < 90
    -> ORDER BY    attendance_pct ASC;
Empty set (0.00 sec)
```

Fig 7.5: Aggregate function queries (1 of 4)

```
mysql> SELECT      g.g_name       AS colour_group,
    ->             sc.`day`,
    ->             a.a_name       AS activity,
    ->             COUNT(p.p_id)  AS enrolled,
    ->             SUM(p.status = 'absent') AS absent
    -> FROM        bootstrap_group g
    -> JOIN        schedule        sc ON sc.g_id = g.g_id
    -> JOIN        activity        a  ON a.a_id  = sc.a_id
    -> JOIN        student         s  ON s.g_id  = g.g_id
    -> JOIN        participates    p  ON p.s_id  = s.s_id AND p.a_id = a.a_id
    -> GROUP BY    g.g_id, g.g_name, sc.`day`, a.a_id, a.a_name
    -> ORDER BY    g.g_name, sc.`day`;
+--------------+-----+--------------------------+----------+--------+
| colour_group | day | activity                 | enrolled | absent |
+--------------+-----+--------------------------+----------+--------+
| Blue         |   1 | Prompt Engineering       |       10 |      0 |
| Blue         |   2 | Design Thinking          |       10 |      0 |
| Blue         |   3 | Biotechnology            |       10 |      0 |
| Blue         |   4 | Electrical Circuits      |       10 |      0 |
| Blue         |   5 | Automobiles and Robotics |       10 |      0 |
| Green        |   1 | Design Thinking          |       10 |      0 |
| Green        |   2 | Biotechnology            |       10 |      0 |
| Green        |   3 | Electrical Circuits      |       10 |      0 |
| Green        |   4 | Automobiles and Robotics |       10 |      0 |
| Green        |   5 | Prompt Engineering       |       10 |      0 |
| Purple       |   1 | Biotechnology            |       10 |      0 |
| Purple       |   2 | Electrical Circuits      |       10 |      0 |
| Purple       |   3 | Automobiles and Robotics |       10 |      0 |
| Purple       |   4 | Prompt Engineering       |       10 |      0 |
| Purple       |   5 | Design Thinking          |       10 |      0 |
| Red          |   1 | Automobiles and Robotics |       10 |      0 |
| Red          |   2 | Prompt Engineering       |       10 |      0 |
| Red          |   3 | Design Thinking          |       10 |      0 |
| Red          |   4 | Biotechnology            |       10 |      0 |
| Red          |   5 | Electrical Circuits      |       10 |      0 |
| Yellow       |   1 | Electrical Circuits      |       10 |      5 |
| Yellow       |   2 | Automobiles and Robotics |       10 |      5 |
| Yellow       |   3 | Prompt Engineering       |       10 |      5 |
| Yellow       |   4 | Design Thinking          |       10 |      5 |
| Yellow       |   5 | Biotechnology            |       10 |      5 |
+--------------+-----+--------------------------+----------+--------+
25 rows in set (0.00 sec)
```

Fig 7.6: Aggregate function queries (2 of 4)

```
mysql> SELECT      a.a_name,
    ->             COUNT(f.f_id)        AS responses,
    ->             ROUND(AVG(f.rating), 2) AS avg_rating,
    ->             MIN(f.rating)        AS worst,
    ->             MAX(f.rating)        AS best
    -> FROM        activity a
    -> LEFT JOIN   feedback f ON f.a_id = a.a_id
    -> GROUP BY    a.a_id, a.a_name
    -> ORDER BY    avg_rating DESC;
+--------------------------+-----------+------------+-------+------+
| a_name                   | responses | avg_rating | worst | best |
+--------------------------+-----------+------------+-------+------+
| Biotechnology            |        15 |       5.00 |     5 |    5 |
| Design Thinking          |        14 |       4.00 |     4 |    4 |
| Prompt Engineering       |        15 |       3.00 |     3 |    3 |
| Automobiles and Robotics |        15 |       2.00 |     2 |    2 |
| Electrical Circuits      |        16 |       1.00 |     1 |    1 |
+--------------------------+-----------+------------+-------+------+
5 rows in set (0.00 sec)
```

Fig 7.7: Aggregate function queries (3 of 4)

```
mysql> SELECT      COUNT(*)                                        AS total_sessions,
    ->             SUM(status = 'present')                         AS present,
    ->             SUM(status = 'late')                            AS late_arrivals,
    ->             SUM(status = 'absent')                          AS absent,
    ->             SUM(status = 'excused')                         AS excused,
    ->             ROUND(AVG(TIMESTAMPDIFF(MINUTE, check_in_time,
    ->                                     check_out_time)), 1)    AS avg_minutes
    -> FROM        participates;
+----------------+---------+---------------+--------+---------+-------------+
| total_sessions | present | late_arrivals | absent | excused | avg_minutes |
+----------------+---------+---------------+--------+---------+-------------+
|            250 |      90 |           135 |     25 |       0 |       432.0 |
+----------------+---------+---------------+--------+---------+-------------+
1 row in set (0.00 sec)
```

Fig 7.8: Aggregate function queries (4 of 4)

7.2 Update Operation

The UPDATE statements below change a mentor's designation, move a session to another room with a joined UPDATE, excuse the absences of one activity using a subquery, and change ratings through the updatable view `vw_circuits_feedback`. Triggers fire on these updates: every attendance status change is written to `attendance_log` by `trg_part_audit`.

```
mysql> UPDATE  mentor
    -> SET     designation = 'Professor'
    -> WHERE   m_name = 'Anitha Rao';
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

Fig 7.9: UPDATE operations (1 of 4)

```
mysql> UPDATE  schedule sc
    -> JOIN    activity a ON a.a_id = sc.a_id
    -> SET     sc.room = 'LAB-BLOCK-E'
    -> WHERE   a.a_name = 'Electrical Circuits';
Query OK, 5 rows affected (0.00 sec)
Rows matched: 5  Changed: 5  Warnings: 0
```

Fig 7.10: UPDATE operations (2 of 4)

```
mysql> UPDATE  participates
    -> SET     status = 'excused'
    -> WHERE   status = 'absent'
    ->   AND   a_id = (
    ->             SELECT a_id FROM (
    ->                 SELECT a_id, SUM(status = 'absent') AS n
    ->                 FROM   participates
    ->                 GROUP  BY a_id
    ->                 ORDER  BY n DESC
    ->                 LIMIT  1
    ->             ) AS worst
    ->         );
Query OK, 6 rows affected (0.00 sec)
Rows matched: 6  Changed: 6  Warnings: 0
```

Fig 7.11: UPDATE operations (3 of 4)

```
mysql> UPDATE  v_circuits_feedback
    -> SET     rating = 4
    -> WHERE   rating = 1;
Query OK, 16 rows affected (0.00 sec)
Rows matched: 16  Changed: 16  Warnings: 0
```

Fig 7.12: UPDATE operations (4 of 4)

7.3 Delete Operation

The DELETE statements below remove a mentor who was never assigned to an activity, remove a student, delete the feedback of excused students with a subquery, and clean up orphaned audit rows with a multi-table DELETE (`DELETE l FROM attendance_log l LEFT JOIN Student s ...`). Foreign-key actions decide what happens to related rows: deleting a student cascades to their participation and feedback rows, and deleting an assigned mentor sets the activity's m_id to NULL.

```
mysql> DELETE FROM mentor WHERE m_name = 'Farhan Ahmed';
Query OK, 1 row affected (0.00 sec)
```

Fig 7.13: DELETE operations (1 of 4)

```
mysql> DELETE FROM student WHERE s_usn = '1BM26CS051';
Query OK, 1 row affected (0.00 sec)
```

Fig 7.14: DELETE operations (2 of 4)

```
mysql> DELETE FROM feedback
    -> WHERE  (s_id, a_id) IN (
    ->            SELECT s_id, a_id FROM (
    ->                SELECT s_id, a_id FROM participates WHERE status = 'excused'
    ->            ) AS ex
    ->        );
Query OK, 0 rows affected (0.00 sec)
```

Fig 7.15: DELETE operations (3 of 4)

```
mysql> DELETE l
    -> FROM   attendance_log l
    -> LEFT JOIN student s ON s.s_id = l.s_id
    -> WHERE  s.s_id IS NULL;
Query OK, 1 row affected (0.00 sec)
```

Fig 7.16: DELETE operations (4 of 4)

7.4 Correlated Query

A correlated subquery refers to a column of the outer query and is evaluated once per outer row. Fig 7.17 uses NOT EXISTS to find students who never submitted feedback. Fig 7.18 uses a correlated scalar subquery to find each student's best-rated activity.

```
SELECT s.s_usn, s.s_name
FROM   Student s
WHERE  NOT EXISTS (SELECT 1 FROM Feedback f WHERE f.s_id = s.s_id)
ORDER  BY s.s_usn;
```

```
mysql> SELECT      s.s_usn, s.s_name
    -> FROM        student s
    -> WHERE       NOT EXISTS (
    ->                 SELECT 1 FROM feedback f WHERE f.s_id = s.s_id
    ->             )
    -> ORDER BY    s.s_usn;
+------------+-------------+
| s_usn      | s_name      |
+------------+-------------+
| 1BM26CV015 | Omkar Joshi |
| 1BM26CV045 | Ujwal Kadam |
+------------+-------------+
2 rows in set (0.00 sec)
```

Fig 7.17: Correlated subqueries (1 of 2)

```
mysql> SELECT      s.s_usn,
    ->             s.s_name,
    ->             (SELECT a.a_name
    ->                FROM feedback f
    ->                JOIN activity a ON a.a_id = f.a_id
    ->               WHERE f.s_id = s.s_id
    ->               ORDER BY f.rating DESC, a.a_name
    ->               LIMIT 1)                AS favourite_activity
    -> FROM        student s
    -> ORDER BY    s.s_usn
    -> LIMIT       15;
+------------+-----------------+--------------------+
| s_usn      | s_name          | favourite_activity |
+------------+-----------------+--------------------+
| 1BM26BT004 | Divya Menon     | Prompt Engineering |
| 1BM26BT009 | Ishita Verma    | Design Thinking    |
| 1BM26BT014 | Nandini Shetty  | Biotechnology      |
| 1BM26BT019 | Sagar Hegde     | Prompt Engineering |
| 1BM26BT024 | Yamini Rao      | Design Thinking    |
| 1BM26BT029 | Deepika Shenoy  | Biotechnology      |
| 1BM26BT034 | Indira Varma    | Prompt Engineering |
| 1BM26BT039 | Nikhil Dsouza   | Design Thinking    |
| 1BM26BT044 | Trisha Baliga   | Biotechnology      |
| 1BM26BT049 | Akhil Ramanna   | Prompt Engineering |
| 1BM26CS001 | Aarav Sharma    | Prompt Engineering |
| 1BM26CS006 | Farhan Siddiqui | Design Thinking    |
| 1BM26CS011 | Kavya Deshpande | Biotechnology      |
| 1BM26CS016 | Pallavi Reddy   | Prompt Engineering |
| 1BM26CS021 | Uday Chandran   | Design Thinking    |
+------------+-----------------+--------------------+
15 rows in set (0.00 sec)
```

Fig 7.18: Correlated subqueries (2 of 2)

7.5 Nested Query

Nested queries use an inner query whose result does not depend on the outer row. Fig 7.19 lists students below the programme's average attendance (scalar subquery). Fig 7.20 lists students who missed Electrical Circuits (IN subquery). Fig 7.21 finds the three activities with the most absences (derived table). Fig 7.22 finds ratings higher than every rating given to Electrical Circuits (> ALL).

```
SELECT s.s_usn, s.s_name, fn_attendance_pct(s.s_id) AS attendance_pct
FROM   Student s
WHERE  fn_attendance_pct(s.s_id) <
       (SELECT AVG(fn_attendance_pct(s2.s_id)) FROM Student s2)
ORDER  BY attendance_pct;
```

```
mysql> SELECT      s.s_usn, s.s_name, fn_attendance_pct(s.s_id) AS attendance_pct
    -> FROM        student s
    -> WHERE       fn_attendance_pct(s.s_id) < (
    ->                 SELECT AVG(fn_attendance_pct(s2.s_id)) FROM student s2
    ->             )
    -> ORDER BY    attendance_pct ASC;
+------------+-----------------+----------------+
| s_usn      | s_name          | attendance_pct |
+------------+-----------------+----------------+
| 1BM26CV005 | Eshan Kulkarni  |          40.00 |
| 1BM26CV015 | Omkar Joshi     |          40.00 |
| 1BM26CV025 | Zoya Fernandes  |          40.00 |
| 1BM26CV035 | Jeevan Poojary  |          40.00 |
| 1BM26CV045 | Ujwal Kadam     |          40.00 |
| 1BM26CV010 | Jatin Prakash   |          60.00 |
| 1BM26CV020 | Tanvi Iyer      |          60.00 |
| 1BM26CV030 | Eknath Salian   |          60.00 |
| 1BM26CV040 | Oviya Natarajan |          60.00 |
| 1BM26CV050 | Bindu Shastry   |          60.00 |
+------------+-----------------+----------------+
10 rows in set (0.01 sec)
```

Fig 7.19: Nested subqueries (1 of 4)

```
mysql> SELECT      s_usn, s_name, branch
    -> FROM        student
    -> WHERE       s_id IN (
    ->                 SELECT s_id FROM participates
    ->                 WHERE  a_id = 5 AND status = 'absent'
    ->             )
    -> ORDER BY    s_usn;
+------------+----------------+--------+
| s_usn      | s_name         | branch |
+------------+----------------+--------+
| 1BM26CV005 | Eshan Kulkarni | Civil  |
| 1BM26CV015 | Omkar Joshi    | Civil  |
| 1BM26CV025 | Zoya Fernandes | Civil  |
| 1BM26CV035 | Jeevan Poojary | Civil  |
| 1BM26CV045 | Ujwal Kadam    | Civil  |
+------------+----------------+--------+
5 rows in set (0.00 sec)
```

Fig 7.20: Nested subqueries (2 of 4)

```
mysql> SELECT      t.a_name, t.absences
    -> FROM        (SELECT a.a_name, SUM(p.status = 'absent') AS absences
    ->                FROM activity a
    ->                JOIN participates p ON p.a_id = a.a_id
    ->               GROUP BY a.a_id, a.a_name) AS t
    -> ORDER BY    t.absences DESC
    -> LIMIT       3;
+--------------------------+----------+
| a_name                   | absences |
+--------------------------+----------+
| Automobiles and Robotics |        5 |
| Biotechnology            |        5 |
| Design Thinking          |        5 |
+--------------------------+----------+
3 rows in set (0.00 sec)
```

Fig 7.21: Nested subqueries (3 of 4)

```
mysql> SELECT      DISTINCT a.a_name, f.rating
    -> FROM        feedback f
    -> JOIN        activity a ON a.a_id = f.a_id
    -> WHERE       f.rating > ALL (
    ->                 SELECT rating FROM feedback WHERE a_id = 5
    ->             );
+--------------------------+--------+
| a_name                   | rating |
+--------------------------+--------+
| Automobiles and Robotics |      2 |
| Biotechnology            |      5 |
| Design Thinking          |      4 |
| Prompt Engineering       |      3 |
+--------------------------+--------+
4 rows in set (0.00 sec)
```

Fig 7.22: Nested subqueries (4 of 4)

7.6 Views

Eight views give the application and the reports a stable, simple interface: `vw_timetable`, `vw_student_attendance` (which uses `fn_attendance_pct` and `fn_risk_level`), `vw_activity_stats`, `vw_attendance_audit`, `vw_activity_ranking` (RANK() window function), `vw_circuits_feedback` (an updatable view WITH CHECK OPTION), `vw_student_schedule` and `vw_group_stats`.

```
mysql> CREATE OR REPLACE VIEW v_timetable AS
    -> SELECT      sc.`day`,
    ->             g.g_name       AS colour_group,
    ->             a.a_name       AS activity,
    ->             sc.room,
    ->             sc.start_time,
    ->             sc.end_time,
    ->             m.m_name       AS mentor
    -> FROM        schedule        sc
    -> JOIN        bootstrap_group g ON g.g_id = sc.g_id
    -> JOIN        activity        a ON a.a_id = sc.a_id
    -> LEFT JOIN   mentor          m ON m.m_id = a.m_id;
Query OK, 0 rows affected (0.01 sec)
```

Fig 7.23: Creating the views (1 of 5)

```
mysql> CREATE OR REPLACE VIEW v_student_attendance AS
    -> SELECT      s.s_id,
    ->             s.s_usn,
    ->             s.s_name,
    ->             g.g_name                        AS colour_group,
    ->             s.branch,
    ->             COUNT(p.p_id)                   AS sessions,
    ->             SUM(p.status = 'present')       AS present,
    ->             SUM(p.status = 'late')          AS late_arrivals,
    ->             SUM(p.status = 'absent')        AS absent,
    ->             SUM(p.status = 'excused')       AS excused,
    ->             fn_attendance_pct(s.s_id)       AS attendance_pct
    -> FROM        student          s
    -> JOIN        bootstrap_group  g ON g.g_id = s.g_id
    -> LEFT JOIN   participates     p ON p.s_id = s.s_id
    -> GROUP BY    s.s_id, s.s_usn, s.s_name, g.g_name, s.branch;
Query OK, 0 rows affected (0.01 sec)
```

Fig 7.24: Creating the views (2 of 5)

```
mysql> CREATE OR REPLACE VIEW v_activity_stats AS
    -> SELECT      a.a_id,
    ->             a.a_name,
    ->             a.capacity,
    ->             m.m_name                                    AS mentor,
    ->             COUNT(p.p_id)                               AS enrolled,
    ->             SUM(p.status IN ('present','late'))         AS attended,
    ->             ROUND(100 * SUM(p.status IN ('present','late'))
    ->                       / NULLIF(COUNT(p.p_id), 0), 2)    AS attendance_pct,
    ->             fn_activity_rating(a.a_id)                  AS avg_rating,
    ->             (SELECT COUNT(*) FROM feedback f
    ->               WHERE f.a_id = a.a_id)                    AS responses
    -> FROM        activity     a
    -> LEFT JOIN   mentor       m ON m.m_id = a.m_id
    -> LEFT JOIN   participates p ON p.a_id = a.a_id
    -> GROUP BY    a.a_id, a.a_name, a.capacity, m.m_name;
Query OK, 0 rows affected (0.01 sec)
```

Fig 7.25: Creating the views (3 of 5)

```
mysql> CREATE OR REPLACE VIEW v_attendance_audit AS
    -> SELECT      l.log_id,
    ->             l.changed_at,
    ->             s.s_usn,
    ->             s.s_name,
    ->             a.a_name    AS activity,
    ->             l.old_status,
    ->             l.new_status,
    ->             l.changed_by
    -> FROM        attendance_log l
    -> JOIN        student        s ON s.s_id = l.s_id
    -> JOIN        activity       a ON a.a_id = l.a_id;
Query OK, 0 rows affected (0.01 sec)
```

Fig 7.26: Creating the views (4 of 5)

```
mysql> CREATE OR REPLACE VIEW v_circuits_feedback AS
    -> SELECT  f_id, s_id, a_id, rating, comments, `time`
    -> FROM    feedback
    -> WHERE   a_id = 5
    -> WITH CHECK OPTION;
Query OK, 0 rows affected (0.00 sec)
```

Fig 7.27: Creating the views (5 of 5)

```
mysql> SELECT * FROM v_timetable ORDER BY `day`, colour_group;
+-----+--------------+--------------------------+-------------+------------+----------+---------------+
| day | colour_group | activity                 | room        | start_time | end_time | mentor        |
+-----+--------------+--------------------------+-------------+------------+----------+---------------+
|   1 | Blue         | Prompt Engineering       | ROOM-02     | 09:00:00   | 16:30:00 | Priya Iyer    |
|   1 | Green        | Design Thinking          | ROOM-03     | 09:00:00   | 16:30:00 | Suresh Pai    |
|   1 | Purple       | Biotechnology            | ROOM-04     | 09:00:00   | 16:30:00 | Sneha Kamath  |
|   1 | Red          | Automobiles and Robotics | ROOM-01     | 09:00:00   | 16:30:00 | Anitha Rao    |
|   1 | Yellow       | Electrical Circuits      | LAB-BLOCK-E | 09:00:00   | 16:30:00 | Ramesh Naik   |
|   2 | Blue         | Design Thinking          | ROOM-03     | 09:00:00   | 16:30:00 | Suresh Pai    |
|   2 | Green        | Biotechnology            | ROOM-04     | 09:00:00   | 16:30:00 | Sneha Kamath  |
|   2 | Purple       | Electrical Circuits      | LAB-BLOCK-E | 09:00:00   | 16:30:00 | Ramesh Naik   |
|   2 | Red          | Prompt Engineering       | ROOM-02     | 09:00:00   | 16:30:00 | Priya Iyer    |
|   2 | Yellow       | Automobiles and Robotics | ROOM-01     | 09:00:00   | 16:30:00 | Anitha Rao    |
|   3 | Blue         | Biotechnology            | ROOM-04     | 09:00:00   | 16:30:00 | Sneha Kamath  |
|   3 | Green        | Electrical Circuits      | LAB-BLOCK-E | 09:00:00   | 16:30:00 | Ramesh Naik   |
|   3 | Purple       | Automobiles and Robotics | ROOM-01     | 09:00:00   | 16:30:00 | Anitha Rao    |
|   3 | Red          | Design Thinking          | ROOM-03     | 09:00:00   | 16:30:00 | Suresh Pai    |
|   3 | Yellow       | Prompt Engineering       | ROOM-02     | 09:00:00   | 16:30:00 | Priya Iyer    |
|   4 | Blue         | Electrical Circuits      | LAB-BLOCK-E | 09:00:00   | 16:30:00 | Ramesh Naik   |
|   4 | Green        | Automobiles and Robotics | ROOM-01     | 09:00:00   | 16:30:00 | Anitha Rao    |
|   4 | Purple       | Prompt Engineering       | ROOM-02     | 09:00:00   | 16:30:00 | Priya Iyer    |
|   4 | Red          | Biotechnology            | ROOM-04     | 09:00:00   | 16:30:00 | Sneha Kamath  |
|   4 | Yellow       | Design Thinking          | ROOM-03     | 09:00:00   | 16:30:00 | Suresh Pai    |
|   5 | Blue         | Automobiles and Robotics | ROOM-01     | 09:00:00   | 16:30:00 | Anitha Rao    |
|   5 | Green        | Prompt Engineering       | ROOM-02     | 09:00:00   | 16:30:00 | Priya Iyer    |
|   5 | Purple       | Design Thinking          | ROOM-03     | 09:00:00   | 16:30:00 | Suresh Pai    |
|   5 | Red          | Electrical Circuits      | LAB-BLOCK-E | 09:00:00   | 16:30:00 | Ramesh Naik   |
|   5 | Yellow       | Biotechnology            | ROOM-04     | 09:00:00   | 16:30:00 | Sneha Kamath  |
+-----+--------------+--------------------------+-------------+------------+----------+---------------+
25 rows in set (0.00 sec)
```

Fig 7.28: Querying the views (1 of 5)

```
mysql> SELECT      s_usn, s_name, colour_group, attendance_pct
    -> FROM        v_student_attendance
    -> WHERE       attendance_pct < 80
    -> ORDER BY    attendance_pct ASC;
+-------------+------------------+---------------+-----------------+
| s_usn       | s_name           | colour_group  | attendance_pct  |
+-------------+------------------+---------------+-----------------+
| 1BM26CV005  | Eshan Kulkarni   | Yellow        |           40.00 |
| 1BM26CV015  | Omkar Joshi      | Yellow        |           40.00 |
| 1BM26CV025  | Zoya Fernandes   | Yellow        |           40.00 |
| 1BM26CV035  | Jeevan Poojary   | Yellow        |           40.00 |
| 1BM26CV045  | Ujwal Kadam      | Yellow        |           40.00 |
| 1BM26CV010  | Jatin Prakash    | Yellow        |           60.00 |
| 1BM26CV020  | Tanvi Iyer       | Yellow        |           60.00 |
| 1BM26CV030  | Eknath Salian    | Yellow        |           60.00 |
| 1BM26CV040  | Oviya Natarajan  | Yellow        |           60.00 |
| 1BM26CV050  | Bindu Shastry    | Yellow        |           60.00 |
+-------------+------------------+---------------+-----------------+
10 rows in set (0.01 sec)
```

Fig 7.29: Querying the views (2 of 5)

```
mysql> SELECT * FROM v_activity_stats ORDER BY avg_rating DESC;
+------+--------------------------+----------+--------------+----------+----------+----------------+------------+-----------+
| a_id | a_name                   | capacity | mentor       | enrolled | attended | attendance_pct | avg_rating | responses |
+------+--------------------------+----------+--------------+----------+----------+----------------+------------+-----------+
|    4 | Biotechnology            |       60 | Sneha Kamath |       50 |       45 |          90.00 |       5.00 |        15 |
|    3 | Design Thinking          |       60 | Suresh Pai   |       50 |       45 |          90.00 |       4.00 |        14 |
|    5 | Electrical Circuits      |       25 | Ramesh Naik  |       50 |       45 |          90.00 |       4.00 |        16 |
|    2 | Prompt Engineering       |       60 | Priya Iyer   |       50 |       45 |          90.00 |       3.00 |        15 |
|    1 | Automobiles and Robotics |       60 | Anitha Rao   |       50 |       45 |          90.00 |       2.19 |        16 |
+------+--------------------------+----------+--------------+----------+----------+----------------+------------+-----------+
5 rows in set (0.00 sec)
```

Fig 7.30: Querying the views (3 of 5)

```
mysql> SELECT      va.a_name,
    ->             va.attendance_pct,
    ->             va.avg_rating,
    ->             COUNT(DISTINCT vt.colour_group) AS groups_hosted
    -> FROM        v_activity_stats va
    -> JOIN        v_timetable      vt ON vt.activity = va.a_name
    -> GROUP BY    va.a_name, va.attendance_pct, va.avg_rating
    -> ORDER BY    va.attendance_pct DESC;
+--------------------------+----------------+------------+----------------+
| a_name                   | attendance_pct | avg_rating | groups_hosted  |
+--------------------------+----------------+------------+----------------+
| Automobiles and Robotics |          90.00 |       2.19 |              5 |
| Biotechnology            |          90.00 |       5.00 |              5 |
| Design Thinking          |          90.00 |       4.00 |              5 |
| Electrical Circuits      |          90.00 |       4.00 |              5 |
| Prompt Engineering       |          90.00 |       3.00 |              5 |
+--------------------------+----------------+------------+----------------+
5 rows in set (0.00 sec)
```

Fig 7.31: Querying the views (4 of 5)

```
mysql> SELECT * FROM v_attendance_audit ORDER BY changed_at DESC LIMIT 20;
+--------+---------------------+-----------+-----------------+------------------------+------------+------------+----------------+
| log_id | changed_at          | s_usn     | s_name          | activity               | old_status | new_status | changed_by     |
+--------+---------------------+-----------+-----------------+------------------------+------------+------------+----------------+
|    226 | 2026-10-03 22:43:18 | 1BM26CV005 | Eshan Kulkarni  | Automobiles and Robotics | absent     | excused    | root@localhost |
|    227 | 2026-10-03 22:43:18 | 1BM26CV015 | Omkar Joshi     | Automobiles and Robotics | absent     | excused    | root@localhost |
|    228 | 2026-10-03 22:43:18 | 1BM26CV025 | Zoya Fernandes  | Automobiles and Robotics | absent     | excused    | root@localhost |
|    229 | 2026-10-03 22:43:18 | 1BM26CV035 | Jeevan Poojary  | Automobiles and Robotics | absent     | excused    | root@localhost |
|    230 | 2026-10-03 22:43:18 | 1BM26CV045 | Ujwal Kadam     | Automobiles and Robotics | absent     | excused    | root@localhost |
|      1 | 2026-10-03 22:40:20 | 1BM26CS001 | Aarav Sharma    | Automobiles and Robotics | absent     | late       | root@localhost |
|      2 | 2026-10-03 22:40:20 | 1BM26CS006 | Farhan Siddiqui | Automobiles and Robotics | absent     | late       | root@localhost |
|      3 | 2026-10-03 22:40:20 | 1BM26CS011 | Kavya Deshpande | Automobiles and Robotics | absent     | late       | root@localhost |
|      4 | 2026-10-03 22:40:20 | 1BM26CS016 | Pallavi Reddy   | Automobiles and Robotics | absent     | late       | root@localhost |
|      5 | 2026-10-03 22:40:20 | 1BM26CS021 | Uday Chandran   | Automobiles and Robotics | absent     | late       | root@localhost |
|      6 | 2026-10-03 22:40:20 | 1BM26CS026 | Aditya Rane     | Automobiles and Robotics | absent     | late       | root@localhost |
|      7 | 2026-10-03 22:40:20 | 1BM26CS031 | Fiza Mirza      | Automobiles and Robotics | absent     | late       | root@localhost |
|      8 | 2026-10-03 22:40:20 | 1BM26CS036 | Keerthi Suresh  | Automobiles and Robotics | absent     | late       | root@localhost |
|      9 | 2026-10-03 22:40:20 | 1BM26CS041 | Pranav Hebbar   | Automobiles and Robotics | absent     | late       | root@localhost |
|     10 | 2026-10-03 22:40:20 | 1BM26CS046 | Vaishnavi Prabhu | Automobiles and Robotics | absent     | late       | root@localhost |
|     86 | 2026-10-03 22:40:20 | 1BM26CV010 | Jatin Prakash   | Automobiles and Robotics | absent     | present    | root@localhost |
|     87 | 2026-10-03 22:40:20 | 1BM26CV020 | Tanvi Iyer      | Automobiles and Robotics | absent     | present    | root@localhost |
|     88 | 2026-10-03 22:40:20 | 1BM26CV030 | Eknath Salian   | Automobiles and Robotics | absent     | present    | root@localhost |
|     89 | 2026-10-03 22:40:20 | 1BM26CV040 | Oviya Natarajan | Automobiles and Robotics | absent     | present    | root@localhost |
|     90 | 2026-10-03 22:40:20 | 1BM26CV050 | Bindu Shastry   | Automobiles and Robotics | absent     | present    | root@localhost |
+--------+---------------------+-----------+-----------------+------------------------+------------+------------+----------------+
20 rows in set (0.00 sec)
```

Fig 7.32: Querying the views (5 of 5)

7.7 Transactions

Part 6 of the script demonstrates transaction control: COMMIT when a new mentor is hired and assigned to an activity together, ROLLBACK of a mistaken bulk update, SAVEPOINT with ROLLBACK TO SAVEPOINT to undo only part of a transaction, and a row lock with `SELECT ... FOR UPDATE` under the REPEATABLE READ isolation level.

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> UPDATE participates SET status = 'excused' WHERE status = 'absent';
Query OK, 24 rows affected (0.00 sec)
Rows matched: 24  Changed: 24  Warnings: 0

mysql>
mysql> SELECT status, COUNT(*) FROM participates GROUP BY status;
+---------+----------+
| status  | COUNT(*) |
+---------+----------+
| present |       90 |
| late    |      136 |
| excused |       29 |
+---------+----------+
3 rows in set (0.00 sec)

mysql>
mysql> ROLLBACK;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> SELECT status, COUNT(*) FROM participates GROUP BY status;
+---------+----------+
| status  | COUNT(*) |
+---------+----------+
| present |       90 |
| absent  |       24 |
| late    |      136 |
| excused |        5 |
+---------+----------+
4 rows in set (0.00 sec)
```

Fig 7.33: Transactions (1 of 3)

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> UPDATE schedule SET room = 'ANNEXE-1' WHERE `day` = 1 AND g_id = 1;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> SAVEPOINT room_changed;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> UPDATE feedback SET rating = 1;
Query OK, 76 rows affected (0.00 sec)
Rows matched: 76  Changed: 76  Warnings: 0

mysql>
mysql> ROLLBACK TO SAVEPOINT room_changed;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> COMMIT;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> SELECT room FROM schedule WHERE `day` = 1 AND g_id = 1;
+----------+
| room     |
+----------+
| ANNEXE-1 |
+----------+
1 row in set (0.00 sec)

mysql> SELECT ROUND(AVG(rating), 2) AS avg_rating_still_intact FROM feedback;
+-------------------------+
| avg_rating_still_intact |
+-------------------------+
|                    3.62 |
+-------------------------+
1 row in set (0.00 sec)
```

Fig 7.34: Transactions (2 of 3)

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> INSERT INTO mentor (m_name, dept, designation, email, phone)
    -> VALUES ('Nisha Verma', 'Design and Innovation', 'Assistant Professor',
    ->         'nisha.verma@univ.edu', '9845010011');
Query OK, 1 row affected (0.00 sec)

mysql>
mysql> UPDATE activity
    -> SET    m_id = LAST_INSERT_ID()
    -> WHERE  a_name = 'Design Thinking';
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> COMMIT;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> SELECT a.a_name, m.m_name
    -> FROM   activity a JOIN mentor m ON m.m_id = a.m_id
    -> WHERE  a.a_name = 'Design Thinking';
+-----------------+-------------+
| a_name          | m_name      |
+-----------------+-------------+
| Design Thinking | Nisha Verma |
+-----------------+-------------+
1 row in set (0.00 sec)
```

Fig 7.35: Transactions (3 of 3)

7.8 Full-Text Search

Feedback comments are searched with the FULLTEXT index `ft_feedback_comments`, in natural-language mode (for example `crowded equipment lab`) and in boolean mode (`+mentor -rushed`, or the exact phrase `"soldering practice"`). Results are ranked by relevance.

```
mysql> SELECT     f.f_id,
    ->            s.s_name,
    ->            a.a_name,
    ->            f.rating,
    ->            f.comments,
    ->            MATCH(f.comments) AGAINST('crowded equipment lab') AS relevance
    -> FROM       feedback f
    -> JOIN       student  s ON s.s_id = f.s_id
    -> JOIN       activity a ON a.a_id = f.a_id
    -> WHERE      MATCH(f.comments) AGAINST('crowded equipment lab')
    -> ORDER BY   relevance DESC
    -> LIMIT      10;
+------+--------------+-------------------------+--------+-----------------------------------------------------------------------------------+---------------------+
| f_id | s_name       | a_name                  | rating | comments                                                                          | relevance           |
+------+--------------+-------------------------+--------+-----------------------------------------------------------------------------------+---------------------+
|    4 | Nandini Shetty | Biotechnology         |      5 | Lab was too crowded, had to share equipment with three other students for most of the day. | 2.867830276489258 |
|   13 | Nikhil Dsouza  | Design Thinking       |      4 | Lab was too crowded, had to share equipment with three other students for most of the day. | 2.867830276489258 |
|   24 | Pallavi Reddy  | Prompt Engineering    |      3 | Lab was too crowded, had to share equipment with three other students for most of the day. | 2.867830276489258 |
|   32 | Pranav Hebbar  | Automobiles and Robotics |   2 | Lab was too crowded, had to share equipment with three other students for most of the day. | 2.867830276489258 |
|   49 | Qasim Ali      | Automobiles and Robotics |   2 | Lab was too crowded, had to share equipment with three other students for most of the day. | 2.867830276489258 |
|   55 | Lalit Bansal   | Electrical Circuits   |      4 | Lab was too crowded, had to share equipment with three other students for most of the day. | 2.867830276489258 |
|   63 | Manav Gupta    | Electrical Circuits   |      4 | Lab was too crowded, had to share equipment with three other students for most of the day. | 2.867830276489258 |
|   71 | Meghana Rau    | Biotechnology         |      5 | Lab was too crowded, had to share equipment with three other students for most of the day. | 2.867830276489258 |
+------+--------------+-------------------------+--------+-----------------------------------------------------------------------------------+---------------------+
8 rows in set (0.00 sec)
```

Fig 7.36: Full-text search (1 of 5)

```
mysql> SELECT      f.f_id, a.a_name, f.rating, f.comments
    -> FROM        feedback f
    -> JOIN        activity a ON a.a_id = f.a_id
    -> WHERE       MATCH(f.comments)
    ->             AGAINST('+mentor -rushed' IN BOOLEAN MODE)
    -> LIMIT       10;
+------+--------------------------+--------+-----------------------------------------------------------------------------------------+
| f_id | a_name                   | rating | comments                                                                                |
+------+--------------------------+--------+-----------------------------------------------------------------------------------------+
|    1 | Electrical Circuits      |      4 | The mentor explained the theory clearly but we ran short of time near the end.          |
|    9 | Biotechnology            |      5 | The mentor explained the theory clearly but we ran short of time near the end.          |
|   20 | Design Thinking          |      4 | The mentor explained the theory clearly but we ran short of time near the end.          |
|   29 | Prompt Engineering       |      3 | The mentor explained the theory clearly but we ran short of time near the end.          |
|   35 | Biotechnology            |      5 | The mentor explained the theory clearly but we ran short of time near the end.          |
|   39 | Design Thinking          |      4 | The mentor explained the theory clearly but we ran short of time near the end.          |
|   46 | Prompt Engineering       |      3 | The mentor explained the theory clearly but we ran short of time near the end.          |
|   54 | Automobiles and Robotics |      2 | The mentor explained the theory clearly but we ran short of time near the end.          |
|   62 | Automobiles and Robotics |      2 | The mentor explained the theory clearly but we ran short of time near the end.          |
|   68 | Electrical Circuits      |      4 | The mentor explained the theory clearly but we ran short of time near the end.          |
+------+--------------------------+--------+-----------------------------------------------------------------------------------------+
10 rows in set (0.00 sec)
```

Fig 7.37: Full-text search (2 of 5)

```
mysql> SELECT      f.f_id, f.rating, f.comments
    -> FROM        feedback f
    -> WHERE       MATCH(f.comments)
    ->             AGAINST('"soldering practice"' IN BOOLEAN MODE);
+------+--------+-----------------------------------------------------------------------------------------+
| f_id | rating | comments                                                                                |
+------+--------+-----------------------------------------------------------------------------------------+
|    6 |      4 | Excellent hands on session, the soldering practice was easily the best part of the week. |
|   14 |      5 | Excellent hands on session, the soldering practice was easily the best part of the week. |
|   25 |      4 | Excellent hands on session, the soldering practice was easily the best part of the week. |
|   34 |      3 | Excellent hands on session, the soldering practice was easily the best part of the week. |
|   51 |      3 | Excellent hands on session, the soldering practice was easily the best part of the week. |
|   59 |      2 | Excellent hands on session, the soldering practice was easily the best part of the week. |
|   67 |      2 | Excellent hands on session, the soldering practice was easily the best part of the week. |
|   73 |      4 | Excellent hands on session, the soldering practice was easily the best part of the week. |
+------+--------+-----------------------------------------------------------------------------------------+
8 rows in set (0.00 sec)
```

Fig 7.38: Full-text search (3 of 5)

```
mysql> SELECT      a_id,
    ->             a_name,
    ->             description,
    ->             MATCH(a_name, description) AGAINST('prototyping lab safety') AS relevance
    -> FROM        activity
    -> WHERE       MATCH(a_name, description) AGAINST('prototyping lab safety')
    -> ORDER BY    relevance DESC;
+------+----------------+-----------------------------------------------------------+--------------------+
| a_id | a_name         | description                                               | relevance          |
+------+----------------+-----------------------------------------------------------+--------------------+
|    4 | Biotechnology  | Wet lab safety, micropipetting technique and gel runs     | 0.9771181344985962 |
|    3 | Design Thinking| Empathise, ideate, paper prototyping and rapid user testing| 0.4885590672492981 |
+------+----------------+-----------------------------------------------------------+--------------------+
2 rows in set (0.00 sec)
```

Fig 7.39: Full-text search (4 of 5)

```
mysql> CALL sp_search_feedback('robot chassis build');
+------+-----------+----------------+--------------------------+--------+-----------------------------------------------------------------------------+--------------------+
| f_id | s_usn     | s_name         | a_name                   | rating | comments                                                                    | relevance          |
+------+-----------+----------------+--------------------------+--------+-----------------------------------------------------------------------------+--------------------+
|  128 | 1BM26CS001 | Aarav Sharma   | Automobiles and Robotics |      5 | Revised after the showcase, the robot build was outstanding.                |  4.1255497932434408 |
|    8 | 1BM26BT024 | Yamini Rao     | Design Thinking          |      4 | Loved building the robot chassis, would happily spend a second day on this activity. | 1.2307043075561523 |
|   17 | 1BM26BT049 | Akhil Ramanna  | Prompt Engineering       |      3 | Loved building the robot chassis, would happily spend a second day on this activity. | 1.2307043075561523 |
|   19 | 1BM26CS001 | Aarav Sharma   | Prompt Engineering       |      3 | Loved building the robot chassis, would happily spend a second day on this activity. | 1.2307043075561523 |
|   27 | 1BM26CS026 | Aditya Rane    | Automobiles and Robotics |      2 | Loved building the robot chassis, would happily spend a second day on this activity. | 1.2307043075561523 |
|   33 | 1BM26CS046 | Vaishnavi Prabhu | Electrical Circuits    |      4 | Loved building the robot chassis, would happily spend a second day on this activity. | 1.2307043075561523 |
|   38 | 1BM26CV025 | Zoya Fernandes | Prompt Engineering       |      3 | Loved building the robot chassis, would happily spend a second day on this activity. | 1.2307043075561523 |
|   42 | 1BM26CV050 | Bindu Shastry  | Automobiles and Robotics |      2 | Loved building the robot chassis, would happily spend a second day on this activity. | 1.2307043075561523 |
|   44 | 1BM26EC002 | Bhavana Nayak  | Automobiles and Robotics |      2 | Loved building the robot chassis, would happily spend a second day on this activity. | 1.2307043075561523 |
|   50 | 1BM26EC022 | Varsha Pillai  | Electrical Circuits      |      4 | Loved building the robot chassis, would happily spend a second day on this activity. | 1.2307043075561523 |
|   58 | 1BM26EC047 | Yash Thakur    | Biotechnology            |      5 | Loved building the robot chassis, would happily spend a second day on this activity. | 1.2307043075561523 |
|   66 | 1BM26ME023 | Wasim Khan     | Biotechnology            |      5 | Loved building the robot chassis, would happily spend a second day on this activity. | 1.2307043075561523 |
|   75 | 1BM26ME048 | Zara Qureshi   | Design Thinking          |      4 | Loved building the robot chassis, would happily spend a second day on this activity. | 1.2307043075561523 |
+------+-----------+----------------+--------------------------+--------+-----------------------------------------------------------------------------+--------------------+
13 rows in set (0.00 sec)

Query OK, 0 rows affected (0.08 sec)
```

Fig 7.40: Full-text search (5 of 5)

8. STORED PROCEDURES, FUNCTIONS AND TRIGGERS

8.1 Stored Procedures and Functions

Calculations that are needed in many places are stored functions, so that the views, the reports and the application all compute them the same way. Multi-step operations that must succeed or fail as a whole are stored procedures.

Table 8.1: Stored functions

Function	Returns
fn_attendance_pct(s_id)	A student's attendance percentage; late counts as attended
fn_risk_level(s_id)	LOW (≥ 80%), MEDIUM (≥ 60%) or HIGH
fn_activity_rating(a_id)	Average rating of an activity
fn_session_minutes(in, out)	Minutes between check-in and check-out
fn_scheduled_day(g_id, a_id)	The day on which a group meets an activity

Table 8.2: Stored procedures

Procedure	What it does
sp_generate_rotation	Builds the 5 × 5 Latin-square rotation; refuses once attendance exists
sp_register_student	Creates the student, five participation rows and a login in one transaction; an EXIT HANDLER rolls back and re-signals on any error
sp_enroll_student	Enrols a late registrant inside a transaction with rollback on failure
sp_transfer_student	Moves a student to another group with a SELECT ... FOR UPDATE row lock and a SAVEPOINT, and rebuilds their participation rows
sp_check_in, sp_check_out	Time-stamp attendance; the status is derived by a trigger
sp_submit_feedback	Checks enrolment, then inserts or updates a rating with ON DUPLICATE KEY UPDATE
sp_mark_absentees	Uses a cursor to loop over the remaining absentees of an activity and excuse each one
sp_group_report	Daily report for one colour group
sp_search_feedback	Ranked full-text search over feedback comments
sp_assert_account_link	Shared check called by the AppUser triggers

For example, `fn_risk_level` reuses `fn_attendance_pct` to classify a student:

```
CREATE FUNCTION fn_risk_level(p_s_id INT UNSIGNED) RETURNS VARCHAR(10)
READS SQL DATA
BEGIN
    DECLARE v DECIMAL(5,2);
    SET v = fn_attendance_pct(p_s_id);
    RETURN CASE WHEN v >= 80 THEN 'LOW' WHEN v >= 60 THEN 'MEDIUM' ELSE 'HIGH' END;
END//
```

```
mysql> DELIMITER $$
mysql>
mysql> -- ================================================================
mysql> -- SECTION F : STORED FUNCTIONS
mysql> -- ================================================================
mysql>
mysql> -- F1. Minutes a student actually spent in a session.
mysql> CREATE FUNCTION fn_session_minutes(p_in DATETIME, p_out DATETIME)
    -> RETURNS INT
    -> DETERMINISTIC
    -> BEGIN
    ->     IF p_in IS NULL OR p_out IS NULL THEN RETURN NULL;
    ->     END IF;
    ->     RETURN TIMESTAMPDIFF(MINUTE, p_in, p_out);
    -> END$$
Query OK, 0 rows affected (0.00 sec)
```

Fig 8.1: Stored functions (1 of 4)

```
mysql> CREATE FUNCTION fn_attendance_pct(p_s_id INT UNSIGNED)
    -> RETURNS DECIMAL(5,2)
    -> READS SQL DATA
    -> BEGIN
    ->     DECLARE v_total   INT DEFAULT 0;
    ->     DECLARE v_present INT DEFAULT 0;
    ->
    ->     SELECT COUNT(*), IFNULL(SUM(status IN ('present','late')), 0)
    ->       INTO v_total, v_present
    ->       FROM participates
    ->      WHERE s_id = p_s_id;
    ->
    ->     IF v_total = 0 THEN
    ->         RETURN 0.00;
    ->     END IF;
    ->
    ->     RETURN ROUND(v_present * 100.0 / v_total, 2);
    -> END$$
Query OK, 0 rows affected (0.00 sec)
```

Fig 8.2: Stored functions (2 of 4)

```
mysql> CREATE FUNCTION fn_activity_rating(p_a_id TINYINT UNSIGNED)
    -> RETURNS DECIMAL(3,2)
    -> READS SQL DATA
    -> BEGIN
    ->     DECLARE v_avg DECIMAL(3,2) DEFAULT NULL;
    ->
    ->     SELECT ROUND(AVG(rating), 2) INTO v_avg
    ->       FROM feedback
    ->      WHERE a_id = p_a_id;
    ->
    ->     RETURN IFNULL(v_avg, 0.00);
    -> END$$
Query OK, 0 rows affected (0.00 sec)
```

Fig 8.3: Stored functions (3 of 4)

```
mysql> CREATE FUNCTION fn_scheduled_day(p_g_id TINYINT UNSIGNED,
    ->                                  p_a_id TINYINT UNSIGNED)
    -> RETURNS TINYINT
    -> READS SQL DATA
    -> BEGIN
    ->     DECLARE v_day TINYINT DEFAULT NULL;
    ->
    ->     SELECT `day` INTO v_day
    ->       FROM schedule
    ->      WHERE g_id = p_g_id AND a_id = p_a_id
    ->      LIMIT 1;
    ->
    ->     RETURN v_day;
    -> END$$
Query OK, 0 rows affected (0.00 sec)
```

Fig 8.4: Stored functions (4 of 4)

```
mysql> CREATE PROCEDURE sp_generate_rotation(IN p_start TIME, IN p_end TIME)
    -> BEGIN
    ->     DECLARE v_day  TINYINT DEFAULT 1;
    ->     DECLARE v_grp  TINYINT;
    ->     DECLARE v_act  TINYINT;
    ->     DECLARE v_rows INT DEFAULT 0;
    ->
    ->     SELECT COUNT(*) INTO v_rows FROM participates;
    ->     IF v_rows > 0 THEN
    ->         SIGNAL SQLSTATE '45000'
    ->           SET MESSAGE_TEXT = 'Cannot regenerate the rotation while attendance rows exist';
    ->     END IF;
    ->
    ->     DELETE FROM schedule;
    ->
    ->     WHILE v_day <= 5 DO
    ->         SET v_grp = 1;
    ->         WHILE v_grp <= 5 DO
    ->             SET v_act = ((v_grp - 1 + v_day - 1) MOD 5) + 1;
    ->             INSERT INTO schedule (`day`, start_time, end_time, room, a_id, g_id)
    ->             VALUES (v_day, p_start, p_end,
    ->                     CONCAT('ROOM-', LPAD(v_act, 2, '0')),
    ->                     v_act, v_grp);
    ->             SET v_grp = v_grp + 1;
    ->         END WHILE;
    ->         SET v_day = v_day + 1;
    ->     END WHILE;
    -> END$$
Query OK, 0 rows affected (0.01 sec)
```

Fig 8.5: Stored procedures (1 of 8)

```
mysql> CREATE PROCEDURE sp_enroll_student(IN  p_usn    VARCHAR(15),
    ->                                    IN  p_name   VARCHAR(80),
    ->                                    IN  p_email  VARCHAR(120),
    ->                                    IN  p_branch VARCHAR(60),
    ->                                    IN  p_sem    TINYINT UNSIGNED,
    ->                                    IN  p_g_id   TINYINT UNSIGNED,
    ->                                    OUT p_s_id   INT UNSIGNED)
    -> BEGIN
    ->     DECLARE EXIT HANDLER FOR SQLEXCEPTION
    ->     BEGIN
    ->         ROLLBACK;
    ->         RESIGNAL;
    ->     END;
    ->
    ->     START TRANSACTION;
    ->
    ->     INSERT INTO student (s_usn, s_name, email, branch, semester, g_id)
    ->     VALUES (p_usn, p_name, p_email, p_branch, p_sem, p_g_id);
    ->
    ->     SET p_s_id = LAST_INSERT_ID();
    ->
    ->     INSERT INTO participates (s_id, a_id, status)
    ->     SELECT p_s_id, a_id, 'absent' FROM activity;
    ->
    ->     COMMIT;
    -> END$$
Query OK, 0 rows affected (0.01 sec)
```

Fig 8.6: Stored procedures (2 of 8)

```
mysql> CREATE PROCEDURE sp_transfer_student(IN p_s_id    INT UNSIGNED,
    ->                                      IN p_new_g_id TINYINT UNSIGNED)
    -> BEGIN
    ->     DECLARE v_old_g TINYINT UNSIGNED DEFAULT NULL;
    ->
    ->     DECLARE EXIT HANDLER FOR SQLEXCEPTION
    ->     BEGIN
    ->         ROLLBACK;
    ->         RESIGNAL;
    ->     END;
    ->
    ->     START TRANSACTION;
    ->
    ->     SELECT g_id INTO v_old_g FROM student WHERE s_id = p_s_id FOR UPDATE;
    ->
    ->     IF v_old_g IS NULL THEN
    ->         ROLLBACK;
    ->         SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'No such student';
    ->     END IF;
    ->
    ->     IF v_old_g = p_new_g_id THEN
    ->         ROLLBACK;
    ->         SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Student is already in that group';
    ->     END IF;
    ->
    ->     UPDATE student SET g_id = p_new_g_id WHERE s_id = p_s_id;
    ->
    ->     SAVEPOINT group_changed;
    ->
    ->     DELETE FROM participates WHERE s_id = p_s_id;
    ->
    ->     INSERT INTO participates (s_id, a_id, status)
    ->     SELECT p_s_id, a_id, 'absent' FROM activity;
    ->
    ->     COMMIT;
    -> END$$
Query OK, 0 rows affected (0.00 sec)
```

Fig 8.7: Stored procedures (3 of 8)

```
mysql> CREATE PROCEDURE sp_check_in(IN p_s_id INT UNSIGNED,
    ->                              IN p_a_id TINYINT UNSIGNED)
    -> BEGIN
    ->     UPDATE participates
    ->        SET check_in_time = NOW(),
    ->            status        = 'present'
    ->      WHERE s_id = p_s_id AND a_id = p_a_id;
    ->
    ->     IF ROW_COUNT() = 0 THEN
    ->         SIGNAL SQLSTATE '45000'
    ->           SET MESSAGE_TEXT = 'No participation row for that student and activity';
    ->     END IF;
    -> END$$
Query OK, 0 rows affected (0.00 sec)
```

Fig 8.8: Stored procedures (4 of 8)

```
mysql> CREATE PROCEDURE sp_check_out(IN p_s_id INT UNSIGNED,
    ->                               IN p_a_id TINYINT UNSIGNED)
    -> BEGIN
    ->     UPDATE participates
    ->        SET check_out_time = NOW()
    ->      WHERE s_id = p_s_id
    ->        AND a_id = p_a_id
    ->        AND check_in_time IS NOT NULL;
    ->
    ->     IF ROW_COUNT() = 0 THEN
    ->         SIGNAL SQLSTATE '45000'
    ->           SET MESSAGE_TEXT = 'Student has not checked in to that activity';
    ->     END IF;
    -> END$$
Query OK, 0 rows affected (0.00 sec)
```

Fig 8.9: Stored procedures (5 of 8)

```
mysql> CREATE PROCEDURE sp_group_report(IN p_g_id TINYINT UNSIGNED)
    -> BEGIN
    ->     SELECT  sc.`day`,
    ->             a.a_name,
    ->             sc.room,
    ->             COUNT(p.p_id)                                   AS enrolled,
    ->             SUM(p.status = 'present')                       AS present,
    ->             SUM(p.status = 'late')                          AS late_arrivals,
    ->             SUM(p.status = 'absent')                        AS absent,
    ->             ROUND(100 * SUM(p.status IN ('present','late'))
    ->                       / COUNT(p.p_id), 2)                   AS attendance_pct,
    ->             fn_activity_rating(a.a_id)                      AS avg_rating
    ->     FROM        schedule     sc
    ->     JOIN        activity     a  ON a.a_id  = sc.a_id
    ->     JOIN        student      st ON st.g_id = sc.g_id
    ->     JOIN        participates p  ON p.s_id  = st.s_id AND p.a_id = a.a_id
    ->     WHERE       sc.g_id = p_g_id
    ->     GROUP BY    sc.`day`, a.a_id, a.a_name, sc.room
    ->     ORDER BY    sc.`day`;
    -> END$$
Query OK, 0 rows affected (0.00 sec)
```

Fig 8.10: Stored procedures (6 of 8)

```
mysql> CREATE PROCEDURE sp_submit_feedback(IN p_s_id     INT UNSIGNED,
    ->                                     IN p_a_id     TINYINT UNSIGNED,
    ->                                     IN p_rating   TINYINT UNSIGNED,
    ->                                     IN p_comments VARCHAR(1000))
    -> BEGIN
    ->     DECLARE v_status VARCHAR(10) DEFAULT NULL;
    ->     DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_status = NULL;
    ->
    ->     SELECT status INTO v_status
    ->       FROM participates
    ->      WHERE s_id = p_s_id AND a_id = p_a_id;
    ->
    ->     IF v_status IS NULL THEN
    ->         SIGNAL SQLSTATE '45000'
    ->           SET MESSAGE_TEXT = 'That student is not enrolled in that activity';
    ->     END IF;
    ->
    ->     INSERT INTO feedback (s_id, a_id, rating, comments)
    ->     VALUES (p_s_id, p_a_id, p_rating, p_comments)
    ->     ON DUPLICATE KEY UPDATE
    ->         rating   = p_rating,
    ->         comments = p_comments,
    ->         `time`   = NOW();
    -> END$$
Query OK, 0 rows affected (0.00 sec)
```

Fig 8.11: Stored procedures (7 of 8)

```
mysql> CREATE PROCEDURE sp_search_feedback(IN p_terms VARCHAR(200))
    -> BEGIN
    ->     SELECT      f.f_id,
    ->                 s.s_usn,
    ->                 s.s_name,
    ->                 a.a_name,
    ->                 f.rating,
    ->                 f.comments,
    ->                 MATCH(f.comments) AGAINST(p_terms IN NATURAL LANGUAGE MODE) AS relevance
    ->     FROM        feedback f
    ->     JOIN        student  s ON s.s_id = f.s_id
    ->     JOIN        activity a ON a.a_id = f.a_id
    ->     WHERE       MATCH(f.comments) AGAINST(p_terms IN NATURAL LANGUAGE MODE)
    ->     ORDER BY    relevance DESC
    ->     LIMIT       20;
    -> END$$
Query OK, 0 rows affected (0.00 sec)
```

Fig 8.12: Stored procedures (8 of 8)

```
mysql> SELECT      s.s_usn,
    ->             s.s_name,
    ->             fn_attendance_pct(s.s_id)                          AS attendance_pct,
    ->             fn_scheduled_day(s.g_id, 5)                        AS circuits_day,
    ->             fn_session_minutes(p.check_in_time, p.check_out_time) AS minutes_in_circuits
    -> FROM        student s
    -> LEFT JOIN   participates p ON p.s_id = s.s_id AND p.a_id = 5
    -> ORDER BY    attendance_pct DESC
    -> LIMIT       15;
+------------+-----------------+----------------+--------------+----------------------+
| s_usn      | s_name          | attendance_pct | circuits_day | minutes_in_circuits  |
+------------+-----------------+----------------+--------------+----------------------+
| 1BM26CS001 | Aarav Sharma    |         100.00 |            5 |                  448 |
| 1BM26EC002 | Bhavana Nayak   |         100.00 |            4 |                  441 |
| 1BM26ME003 | Chirag Patel    |         100.00 |            3 |                  434 |
| 1BM26BT004 | Divya Menon     |         100.00 |            2 |                  427 |
| 1BM26ME018 | Rithika Bhat    |         100.00 |            3 |                  434 |
| 1BM26CS006 | Farhan Siddiqui |         100.00 |            5 |                  448 |
| 1BM26EC007 | Gayathri Kamath |         100.00 |            4 |                  441 |
| 1BM26ME008 | Harish Raghavan |         100.00 |            3 |                  434 |
| 1BM26BT009 | Ishita Verma    |         100.00 |            2 |                  427 |
| 1BM26CS011 | Kavya Deshpande |         100.00 |            5 |                  448 |
| 1BM26EC012 | Lohith Krishnan |         100.00 |            4 |                  441 |
| 1BM26ME013 | Manav Gupta     |         100.00 |            3 |                  434 |
| 1BM26BT014 | Nandini Shetty  |         100.00 |            2 |                  427 |
| 1BM26EC017 | Qasim Ali       |         100.00 |            4 |                  441 |
| 1BM26CS016 | Pallavi Reddy   |         100.00 |            5 |                  448 |
+------------+-----------------+----------------+--------------+----------------------+
15 rows in set (0.02 sec)
```

Fig 8.13: Calling the functions and procedures (1 of 8)

```
mysql> SELECT      a_name, fn_activity_rating(a_id) AS avg_rating
    -> FROM        activity
    -> WHERE       fn_activity_rating(a_id) >= 3.0
    -> ORDER BY    avg_rating DESC;
+----------------------+-------------+
| a_name               | avg_rating  |
+----------------------+-------------+
| Biotechnology        |        5.00 |
| Design Thinking      |        4.00 |
| Electrical Circuits  |        4.00 |
| Prompt Engineering   |        3.00 |
+----------------------+-------------+
4 rows in set (0.00 sec)
```

Fig 8.14: Calling the functions and procedures (2 of 8)

```
mysql> CALL sp_group_report(3);
+-----+-------------------------+-------------+----------+---------+---------------+--------+----------------+------------+
| day | a_name                  | room        | enrolled | present | late_arrivals | absent | attendance_pct | avg_rating |
+-----+-------------------------+-------------+----------+---------+---------------+--------+----------------+------------+
|   1 | Design Thinking         | ROOM-03     |       10 |       0 |            10 |      0 |         100.00 |       4.00 |
|   2 | Biotechnology           | ROOM-04     |       10 |      10 |             0 |      0 |         100.00 |       5.00 |
|   3 | Electrical Circuits     | LAB-BLOCK-E |       10 |       0 |            10 |      0 |         100.00 |       4.00 |
|   4 | Automobiles and Robotics | ROOM-01     |       10 |       0 |            10 |      0 |         100.00 |       2.19 |
|   5 | Prompt Engineering      | ROOM-02     |       10 |      10 |             0 |      0 |         100.00 |       3.00 |
+-----+-------------------------+-------------+----------+---------+---------------+--------+----------------+------------+
5 rows in set (0.00 sec)

Query OK, 0 rows affected (0.02 sec)
```

Fig 8.15: Calling the functions and procedures (3 of 8)

```
mysql> CALL sp_enroll_student('1BM26EC052', 'Rohan Dutta', 'rohan.dutta@univ.edu',
    ->                        'Electronics', 1, 2, @new_student);
Query OK, 0 rows affected (0.01 sec)
```

Fig 8.16: Calling the functions and procedures (4 of 8)

```
mysql> SELECT @new_student AS newly_created_s_id;
+---------------------+
| newly_created_s_id |
+---------------------+
|                  52 |
+---------------------+
1 row in set (0.00 sec)
```

Fig 8.17: Calling the functions and procedures (5 of 8)

```
mysql> CALL sp_check_in(@new_student, 2);
Query OK, 1 row affected (0.00 sec)
```

Fig 8.18: Calling the functions and procedures (6 of 8)

```
mysql> CALL sp_check_out(@new_student, 2);
Query OK, 1 row affected (0.00 sec)
```

Fig 8.19: Calling the functions and procedures (7 of 8)

```
mysql> SELECT * FROM participates WHERE s_id = @new_student;
+------+------+------+---------------------+---------------------+--------+
| p_id | s_id | a_id | check_in_time       | check_out_time      | status |
+------+------+------+---------------------+---------------------+--------+
|  263 |   52 |    1 | NULL                | NULL                | absent |
|  264 |   52 |    2 | 2026-10-03 22:43:19 | 2026-10-03 22:43:19 | late   |
|  265 |   52 |    3 | NULL                | NULL                | absent |
|  266 |   52 |    4 | NULL                | NULL                | absent |
|  267 |   52 |    5 | NULL                | NULL                | absent |
+------+------+------+---------------------+---------------------+--------+
5 rows in set (0.00 sec)
```

Fig 8.20: Calling the functions and procedures (8 of 8)

8.2 Triggers

Twelve triggers enforce the rules that a CHECK constraint cannot express, because they depend on other rows. When a rule is broken, the trigger raises `SIGNAL SQLSTATE '45000'` with a readable message, which the web application shows to the user.

Table 8.3: Triggers

Trigger	Rule it enforces
trg_schedule_room_clash, trg_schedule_room_clash_upd	A room holds only one group per day (on INSERT and UPDATE)
trg_part_validate	The activity must be on the student's group rotation and within capacity
trg_part_set_status	A check-in more than 15 minutes after the start sets the status to late, otherwise present
trg_part_audit	Logs every status change in attendance_log with the person who made it
trg_feedback_validate	Only students who attended an activity may rate it
trg_feedback_clamp	Editing a rating re-stamps its time

Trigger	Rule it enforces
trg_student_protect	A student with recorded attendance cannot be deleted
trg_appuser_link_ins, trg_appuser_link_upd	An account's role must match its linked student or mentor
trg_student_email_sync, trg_mentor_email_sync	The login username follows email changes

The late-arrival rule is implemented by `trg_part_set_status`:

```
CREATE TRIGGER trg_part_set_status
BEFORE UPDATE ON Participates
FOR EACH ROW
BEGIN
    DECLARE v_g_id  TINYINT UNSIGNED;
    DECLARE v_start TIME;

    IF NEW.status <> 'excused'
       AND NEW.check_in_time IS NOT NULL
       AND (OLD.check_in_time IS NULL OR NEW.check_in_time <> OLD.check_in_time) THEN

        SELECT g_id INTO v_g_id FROM Student WHERE s_id = NEW.s_id;
        SELECT start_time INTO v_start FROM Schedule
        WHERE g_id = v_g_id AND a_id = NEW.a_id LIMIT 1;

        IF v_start IS NOT NULL AND TIME(NEW.check_in_time) > ADDTIME(v_start, '00:15:00') THEN
            SET NEW.status = 'late';
        ELSE
            SET NEW.status = 'present';
        END IF;
    END IF;
END//
```

```
mysql> CREATE TRIGGER trg_participates_bi
    -> BEFORE INSERT ON participates
    -> FOR EACH ROW
    -> BEGIN
    ->     DECLARE v_g_id      TINYINT UNSIGNED DEFAULT NULL;
    ->     DECLARE v_scheduled INT DEFAULT 0;
    ->     DECLARE v_cap       SMALLINT UNSIGNED DEFAULT 0;
    ->     DECLARE v_count     INT DEFAULT 0;
    ->
    ->     SELECT g_id INTO v_g_id FROM student WHERE s_id = NEW.s_id;
    ->
    ->     SELECT COUNT(*) INTO v_scheduled
    ->       FROM schedule
    ->      WHERE g_id = v_g_id AND a_id = NEW.a_id;
    ->
    ->     IF v_scheduled = 0 THEN
    ->         SIGNAL SQLSTATE '45000'
    ->           SET MESSAGE_TEXT = 'That activity is not on the rotation for this student''s group';
    ->     END IF;
    ->
    ->     SELECT capacity INTO v_cap FROM activity WHERE a_id = NEW.a_id;
    ->
    ->     SELECT COUNT(*) INTO v_count
    ->       FROM participates p
    ->       JOIN student s ON s.s_id = p.s_id
    ->      WHERE p.a_id = NEW.a_id AND s.g_id = v_g_id;
    ->
    ->     IF v_count >= v_cap THEN
    ->         SIGNAL SQLSTATE '45000'
    ->           SET MESSAGE_TEXT = 'That activity session is already at capacity for this group';
    ->     END IF;
    -> END$$
Query OK, 0 rows affected (0.01 sec)
```

Fig 8.21: Creating the triggers (1 of 6)

```
mysql> CREATE TRIGGER trg_participates_bu
    -> BEFORE UPDATE ON participates
    -> FOR EACH ROW
    -> BEGIN
    ->     DECLARE v_g_id  TINYINT UNSIGNED DEFAULT NULL;
    ->     DECLARE v_start TIME DEFAULT NULL;
    ->
    ->     IF NEW.status <> 'excused'
    ->        AND NEW.check_in_time IS NOT NULL
    ->        AND (OLD.check_in_time IS NULL
    ->             OR NEW.check_in_time <> OLD.check_in_time) THEN
    ->
    ->         SELECT g_id INTO v_g_id FROM student WHERE s_id = NEW.s_id;
    ->
    ->         SELECT start_time INTO v_start
    ->           FROM schedule
    ->          WHERE g_id = v_g_id AND a_id = NEW.a_id
    ->          LIMIT 1;
    ->
    ->         IF v_start IS NOT NULL
    ->            AND TIME(NEW.check_in_time) > ADDTIME(v_start, '00:15:00') THEN
    ->             SET NEW.status = 'late';
    ->         ELSE
    ->             SET NEW.status = 'present';
    ->         END IF;
    ->     END IF;
    -> END$$
Query OK, 0 rows affected (0.01 sec)
```

Fig 8.22: Creating the triggers (2 of 6)

```
mysql> CREATE TRIGGER trg_participates_au
    -> AFTER UPDATE ON participates
    -> FOR EACH ROW
    -> BEGIN
    ->     IF NEW.status <> OLD.status THEN
    ->         INSERT INTO attendance_log
    ->             (p_id, s_id, a_id, old_status, new_status, changed_by)
    ->         VALUES
    ->             (NEW.p_id, NEW.s_id, NEW.a_id,
    ->              OLD.status, NEW.status, CURRENT_USER());
    ->     END IF;
    -> END$$
Query OK, 0 rows affected (0.01 sec)
```

Fig 8.23: Creating the triggers (3 of 6)

```
mysql> CREATE TRIGGER trg_feedback_bi
    -> BEFORE INSERT ON feedback
    -> FOR EACH ROW
    -> BEGIN
    ->     DECLARE v_status VARCHAR(10) DEFAULT NULL;
    ->     DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_status = NULL;
    ->
    ->     SELECT status INTO v_status
    ->       FROM participates
    ->      WHERE s_id = NEW.s_id AND a_id = NEW.a_id;
    ->
    ->     IF v_status IS NULL THEN
    ->         SIGNAL SQLSTATE '45000'
    ->           SET MESSAGE_TEXT = 'Student did not participate in this activity';
    ->     END IF;
    ->
    ->     IF v_status = 'absent' THEN
    ->         SIGNAL SQLSTATE '45000'
    ->           SET MESSAGE_TEXT = 'An absent student cannot rate the activity';
    ->     END IF;
    -> END$$
Query OK, 0 rows affected (0.01 sec)
```

Fig 8.24: Creating the triggers (4 of 6)

```
mysql> CREATE TRIGGER trg_schedule_bi
    -> BEFORE INSERT ON schedule
    -> FOR EACH ROW
    -> BEGIN
    ->     DECLARE v_clash INT DEFAULT 0;
    ->
    ->     IF NEW.room IS NOT NULL THEN
    ->         SELECT COUNT(*) INTO v_clash
    ->           FROM schedule
    ->          WHERE `day` = NEW.`day` AND room = NEW.room;
    ->
    ->         IF v_clash > 0 THEN
    ->             SIGNAL SQLSTATE '45000'
    ->               SET MESSAGE_TEXT = 'That room is already booked on that day';
    ->         END IF;
    ->     END IF;
    -> END$$
Query OK, 0 rows affected (0.00 sec)
```

Fig 8.25: Creating the triggers (5 of 6)

```
mysql> CREATE TRIGGER trg_student_bd
    -> BEFORE DELETE ON student
    -> FOR EACH ROW
    -> BEGIN
    ->     DECLARE v_attended INT DEFAULT 0;
    ->
    ->     SELECT COUNT(*) INTO v_attended
    ->       FROM participates
    ->      WHERE s_id = OLD.s_id
    ->        AND status IN ('present','late');
    ->
    ->     IF v_attended > 0 THEN
    ->         SIGNAL SQLSTATE '45000'
    ->           SET MESSAGE_TEXT = 'Cannot delete a student with recorded attendance';
    ->     END IF;
    -> END$$
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;
```

Fig 8.26: Creating the triggers (6 of 6)

```
mysql> UPDATE activity SET capacity = 5 WHERE a_id = 5;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> INSERT INTO participates (s_id, a_id) VALUES (1, 5);
ERROR 1644 (45000): That activity session is already at capacity for this group
mysql> UPDATE activity SET capacity = 25 WHERE a_id = 5;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

Fig 8.27: Triggers firing on invalid operations (1 of 5)

```
mysql> INSERT INTO schedule (`day`, start_time, end_time, room, a_id, g_id)
    -> VALUES (1, '09:00:00', '16:30:00', 'ROOM-01', 2, 1);
ERROR 1062 (23000): Duplicate entry '1-1' for key 'schedule.uq_schedule_group_day'
mysql>
```

Fig 8.28: Triggers firing on invalid operations (2 of 5)

```
mysql> UPDATE participates
    -> SET    check_in_time = TIMESTAMP('2026-10-05', '11:45:00')
    -> WHERE  s_id = 2 AND a_id = 2;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

Fig 8.29: Triggers firing on invalid operations (3 of 5)

```
mysql> SELECT status, check_in_time FROM participates WHERE s_id = 2 AND a_id = 2;
+--------+---------------------+
| status | check_in_time       |
+--------+---------------------+
| late   | 2026-10-05 11:45:00 |
+--------+---------------------+
1 row in set (0.00 sec)
```

Fig 8.30: Triggers firing on invalid operations (4 of 5)

```
mysql> SELECT * FROM v_attendance_audit
    -> WHERE  s_usn = '1BM26EC002'
    -> ORDER  BY changed_at DESC
    -> LIMIT  5;
+--------+---------------------+-----------+---------------+--------------------------+------------+------------+----------------+
| log_id | changed_at          | s_usn     | s_name        | activity                 | old_status | new_status | changed_by     |
+--------+---------------------+-----------+---------------+--------------------------+------------+------------+----------------+
|    257 | 2026-10-03 22:43:19 | 1BM26EC002 | Bhavana Nayak | Prompt Engineering       | present    | late       | root@localhost |
|    191 | 2026-10-03 22:40:20 | 1BM26EC002 | Bhavana Nayak | Automobiles and Robotics | absent     | late       | root@localhost |
|    101 | 2026-10-03 22:40:20 | 1BM26EC002 | Bhavana Nayak | Biotechnology            | absent     | late       | root@localhost |
|     56 | 2026-10-03 22:40:20 | 1BM26EC002 | Bhavana Nayak | Design Thinking          | absent     | late       | root@localhost |
|    146 | 2026-10-03 22:40:20 | 1BM26EC002 | Bhavana Nayak | Electrical Circuits      | absent     | present    | root@localhost |
+--------+---------------------+-----------+---------------+--------------------------+------------+------------+----------------+
5 rows in set (0.00 sec)
```

Fig 8.31: Triggers firing on invalid operations (5 of 5)

9. FRONT END DEVELOPMENT

9.1 Technology Stack

Table 9.1: Technology stack

Layer	Technology	Purpose
Relational database	MySQL 8.0	Records, business rules, statistics and audit log
Graph database	Neo4j (Cypher)	Interests, peer connections, recommendations, shortest paths
Web framework	Next.js 16, React 19, TypeScript	Pages for the user interface; Route Handlers under /api as the backend
Styling	Tailwind CSS 4	Responsive layout of the dashboards
Database clients	mysql2, neo4j-driver	Connection pooling and parameterised queries (? and $params)
Security	bcryptjs, jose, zod	Password hashing, signed session tokens (JWT), input validation
Testing	Vitest	Unit, database-integration and end-to-end HTTP tests

9.2 Architecture

The browser talks only to the Next.js server, which talks to both databases. A request is handled in six steps:

1. A page calls an `/api/...` endpoint and sends the session cookie.
2. A proxy checks the session, redirects users who may not open that page, and rejects cross-site writes.
3. The route handler checks the user's role again, validates the input with zod, and calls a service.
4. The service runs parameterised SQL through mysql2 and/or Cypher through neo4j-driver.
5. A database rule violation, such as a trigger's SIGNAL, becomes a clear error message for the user.
6. When MySQL data changed, that student's part of the graph is refreshed in Neo4j after the response has been sent.

If Neo4j is unreachable, the graph pages show a clear message and every MySQL feature keeps working.

9.3 Screens

Each role has its own dashboard. The figures below show the main screens.

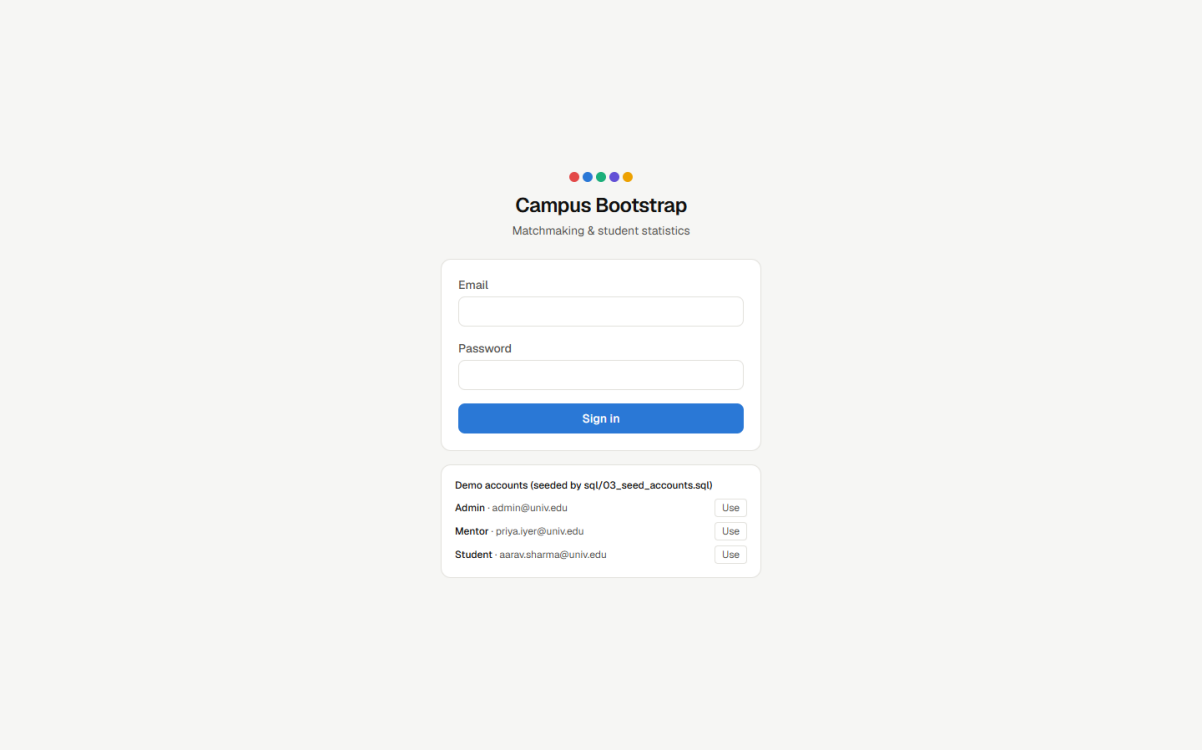


Fig 9.1: Login page

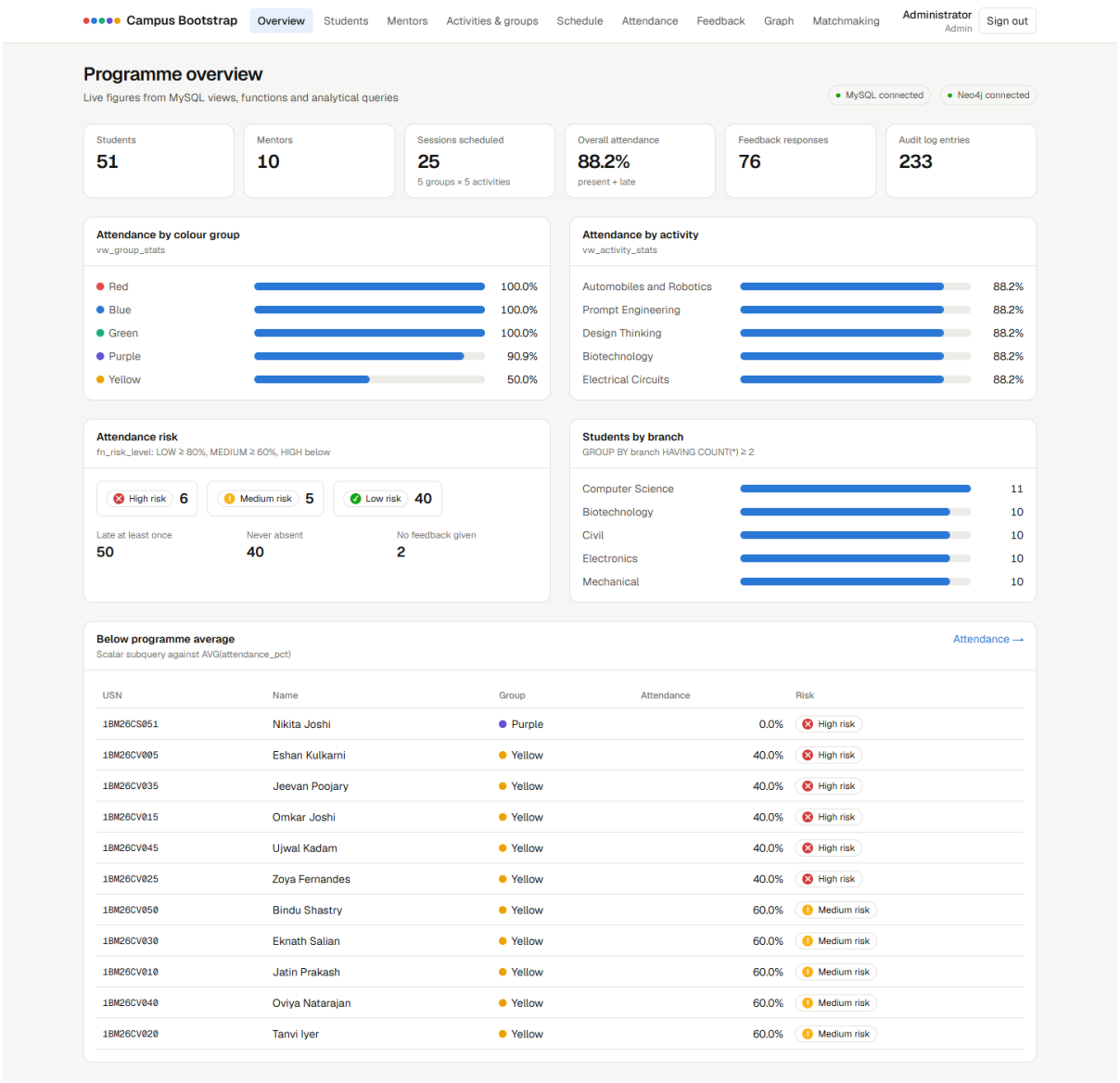


Fig 9.2: Admin: programme overview built from views, functions and analytical queries

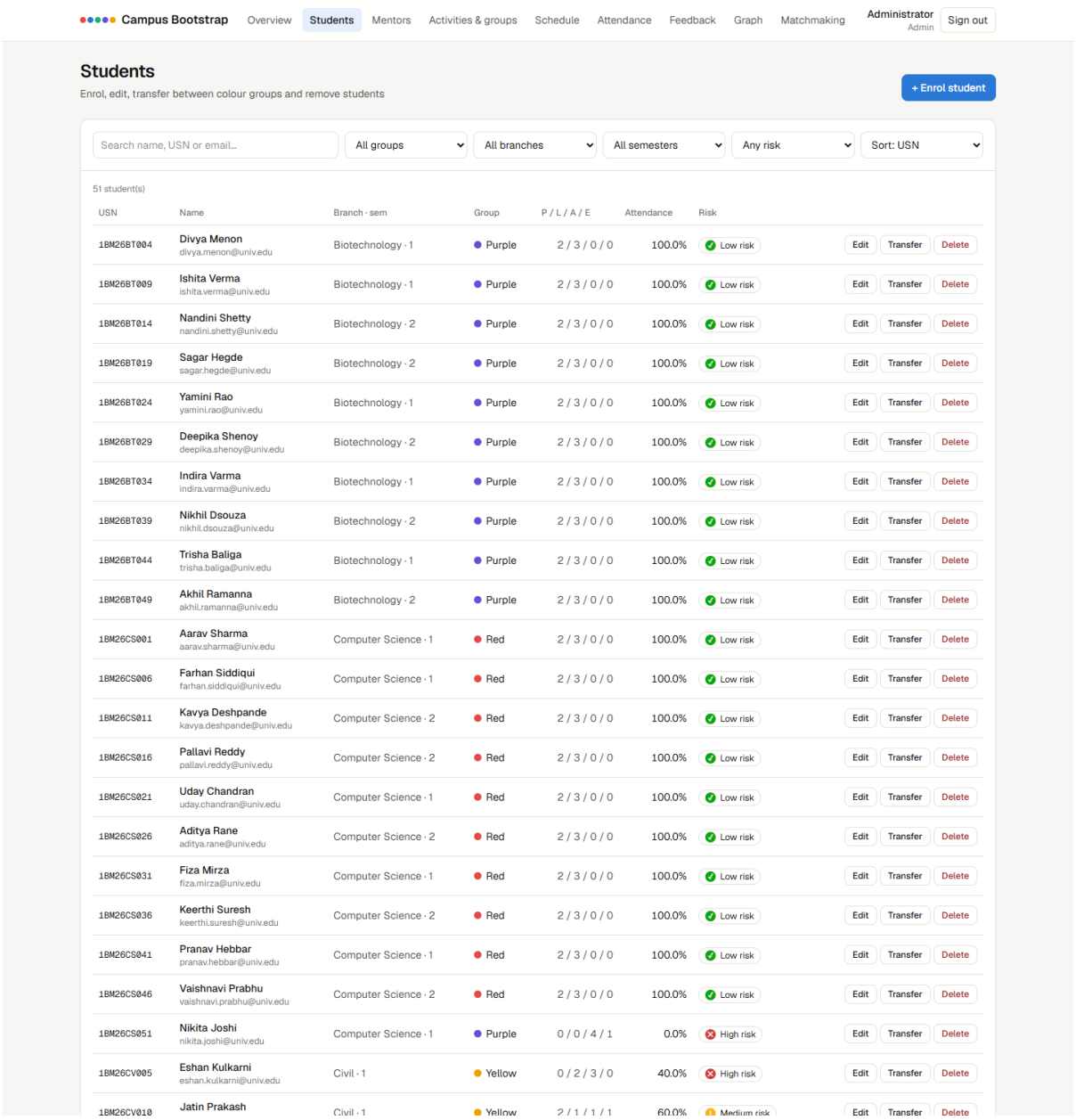


Fig 9.3: Admin: student management (enrol, edit, transfer, delete)

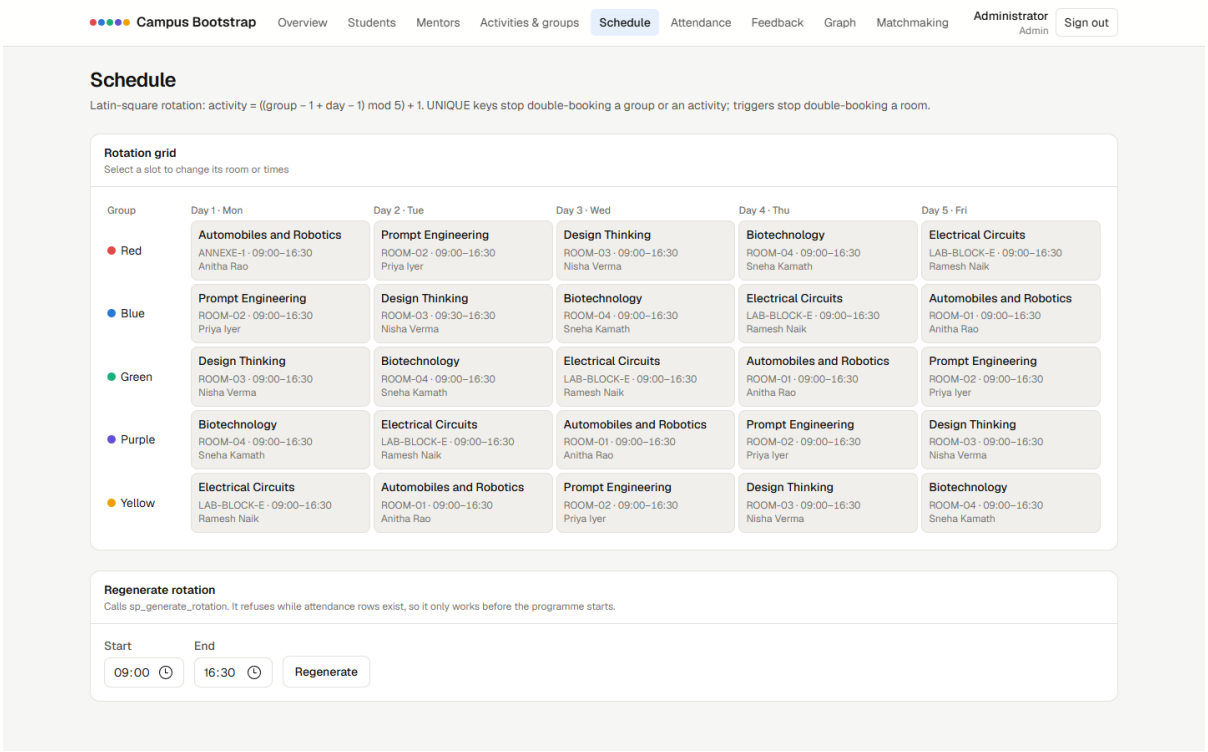


Fig 9.4: Admin: rotation grid

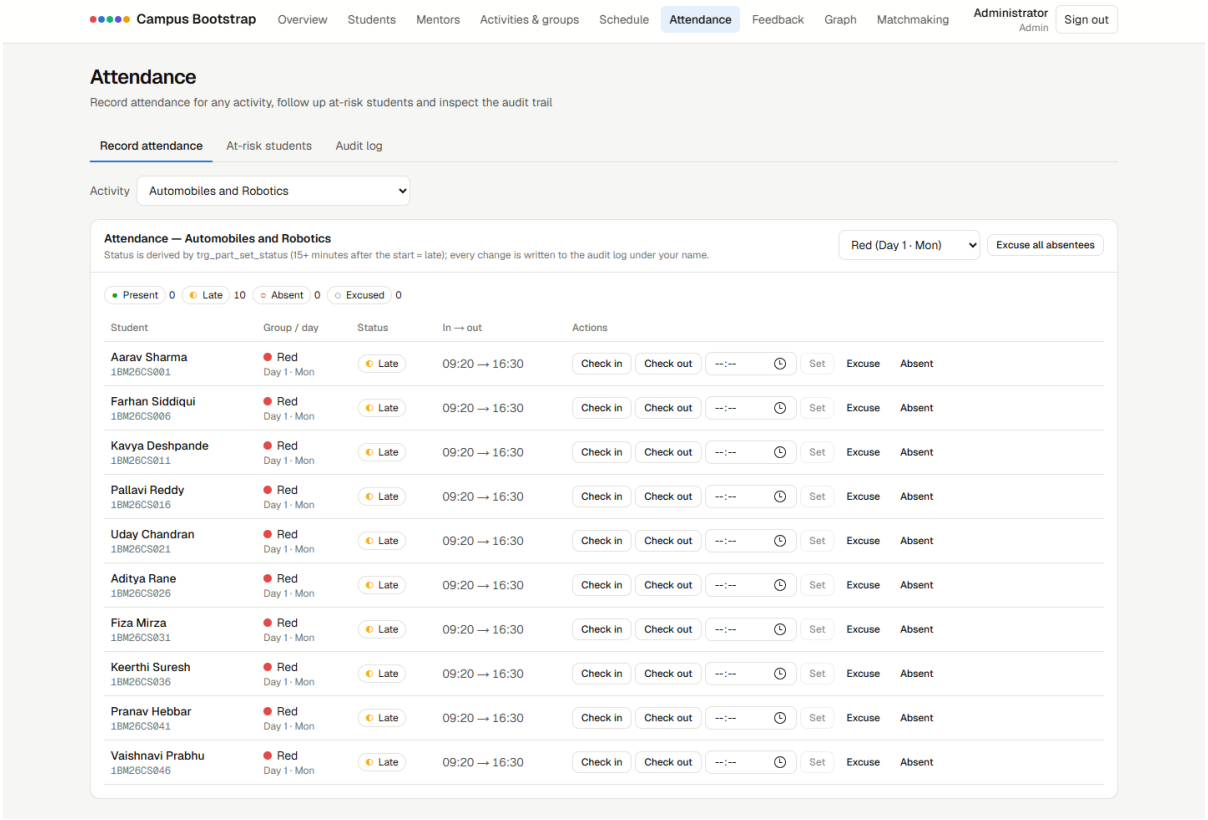


Fig 9.5: Admin: attendance risk report and audit log

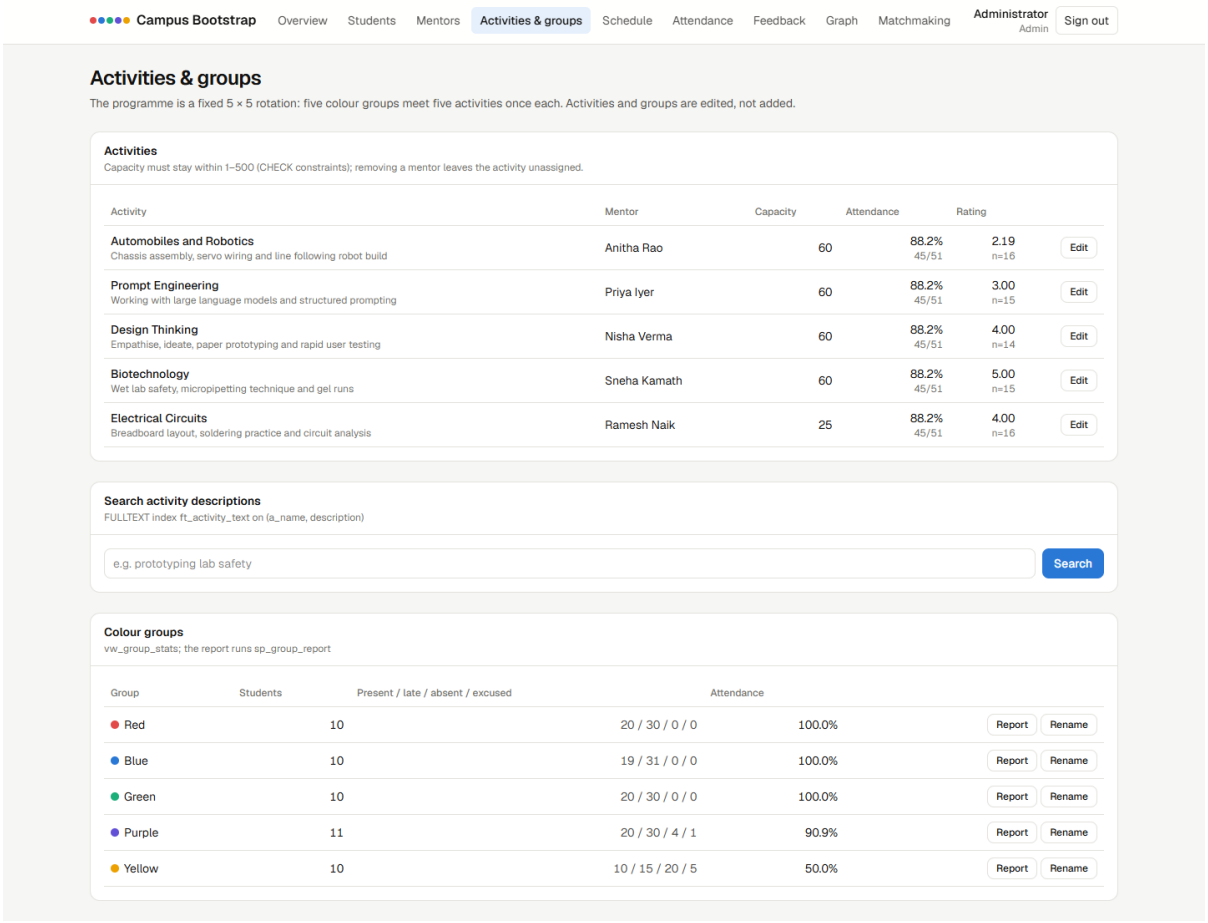


Fig 9.6: Admin: activities

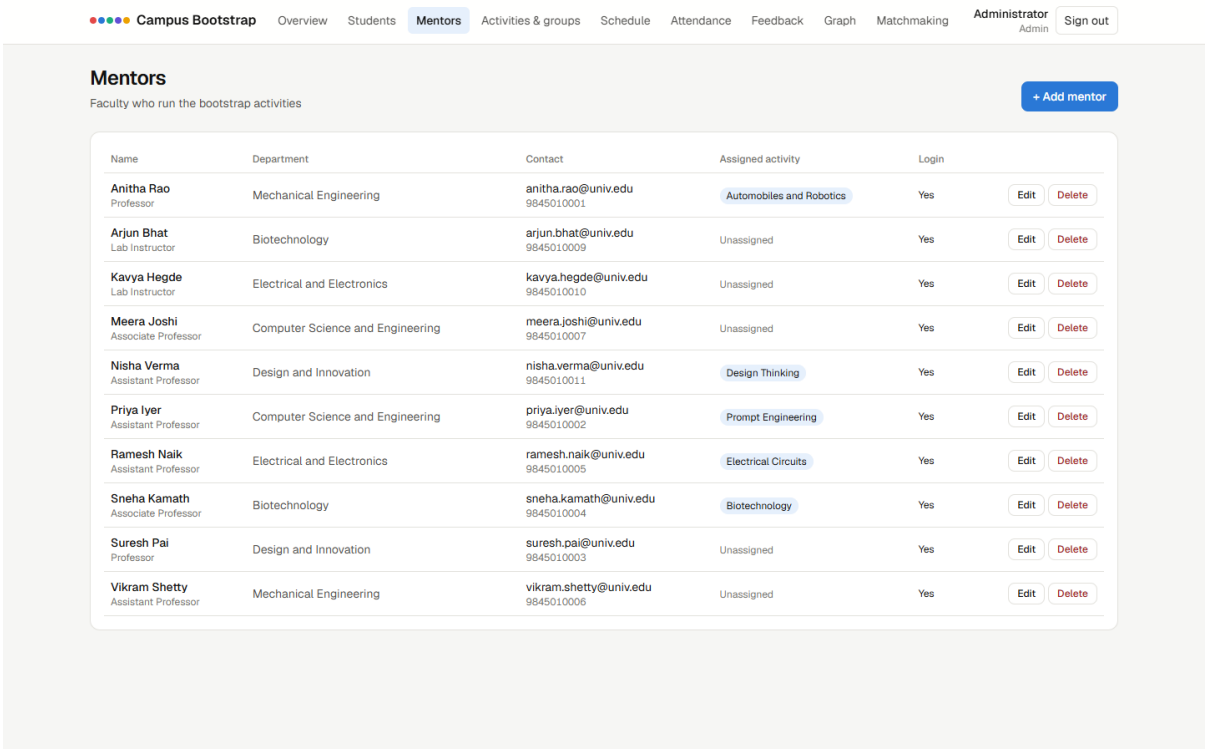


Fig 9.7: Admin: mentors

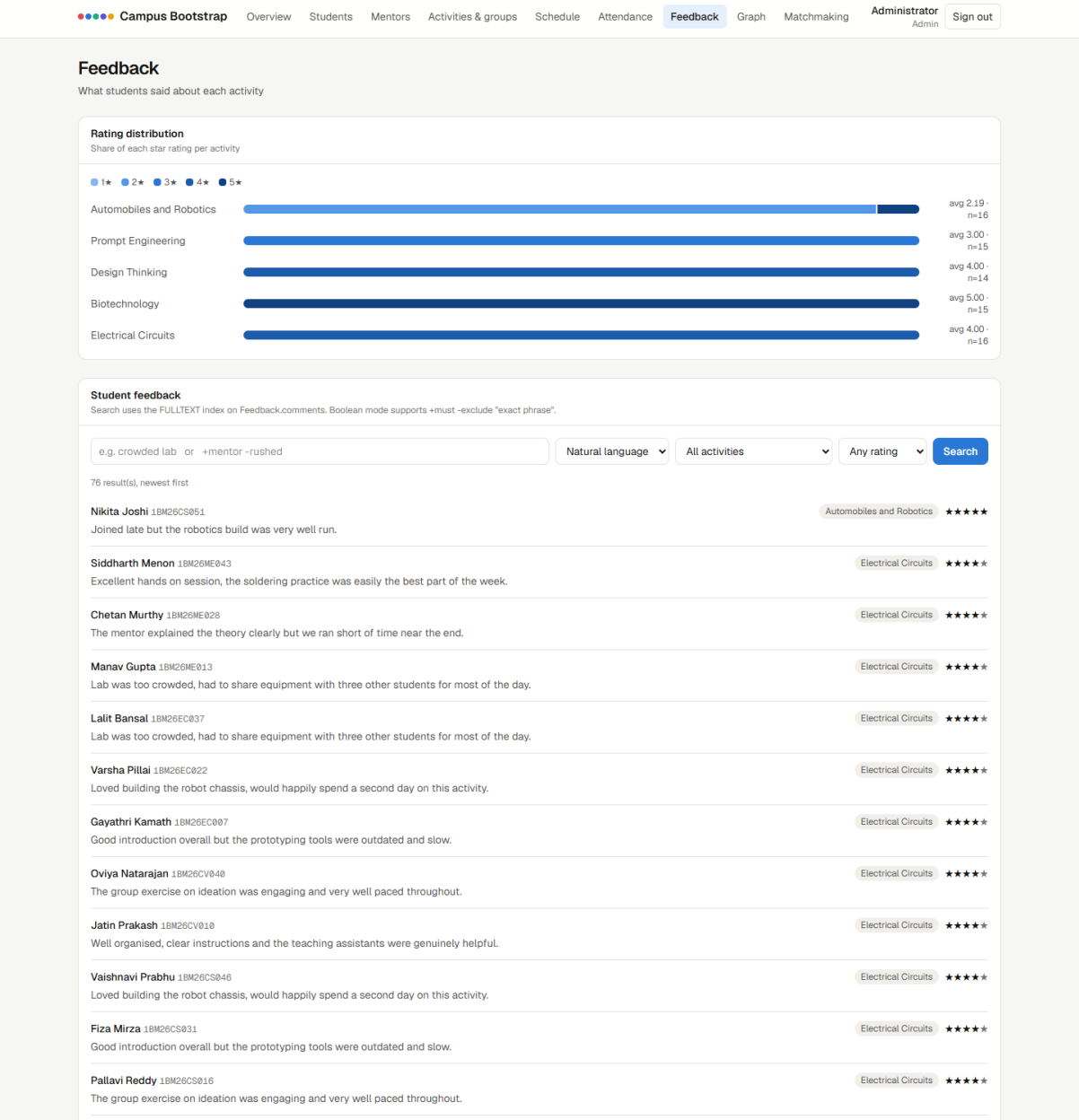


Fig 9.8: Admin: feedback review with full-text search

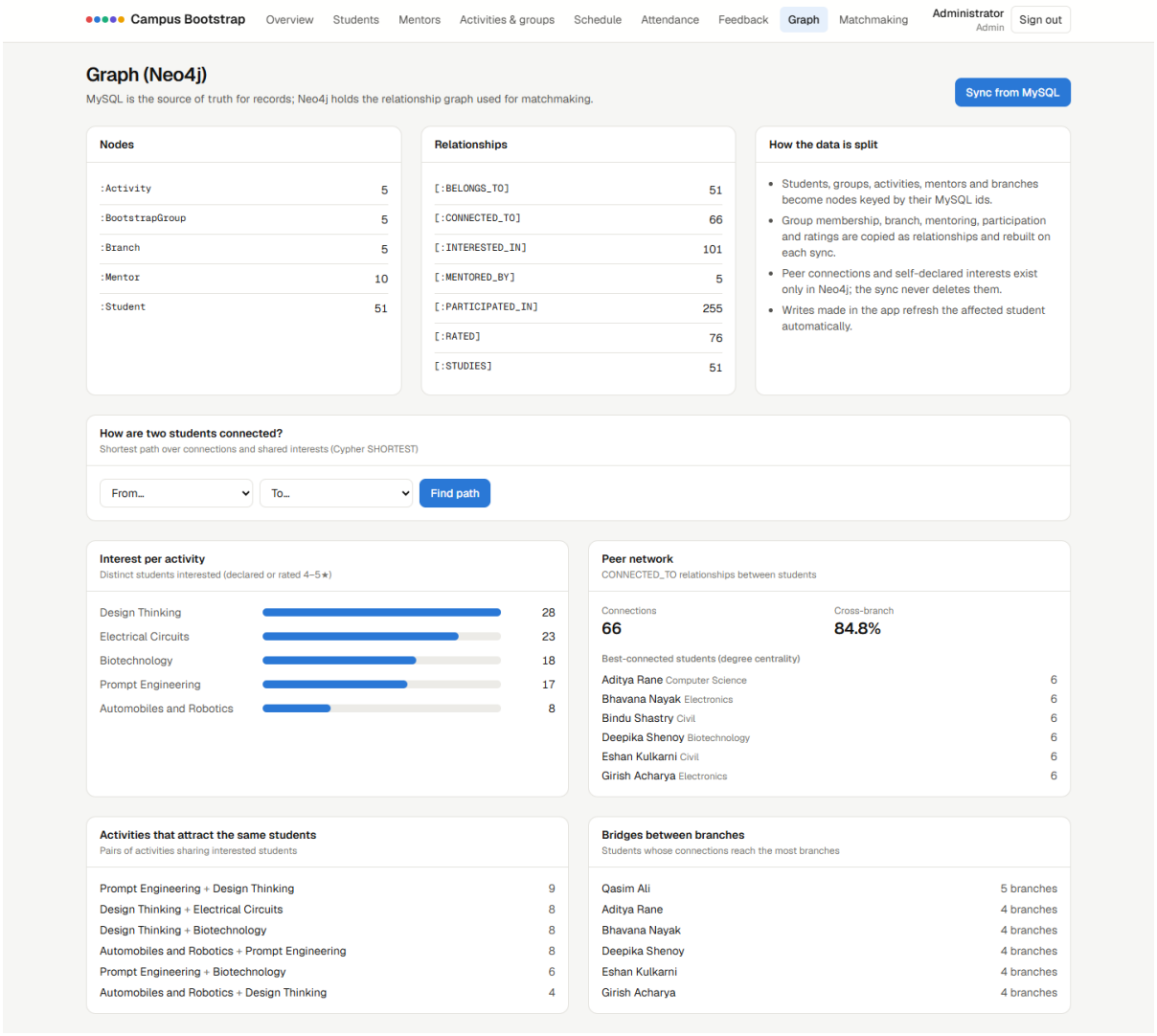


Fig 9.9: Admin: Neo4j graph status and sync

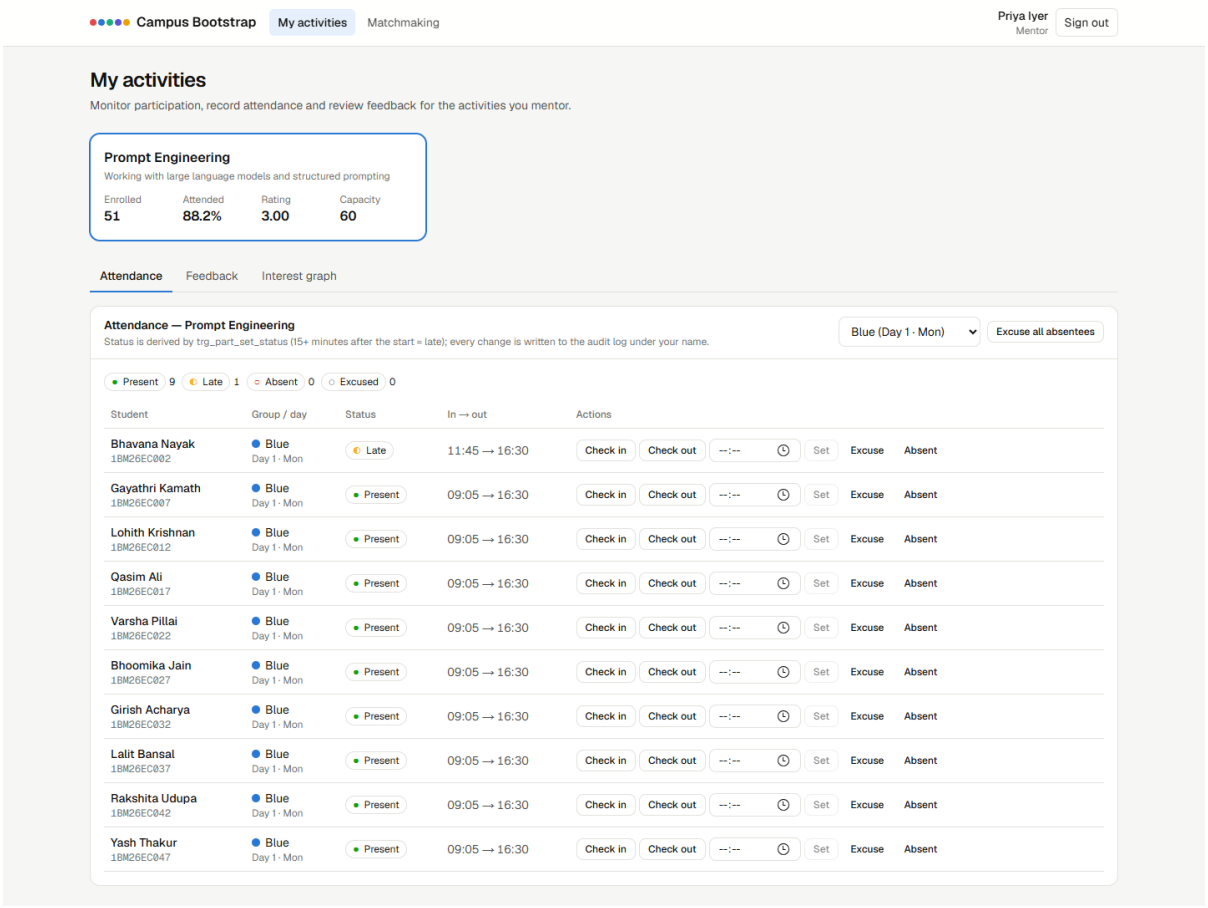


Fig 9.10: Mentor: activities and attendance roster

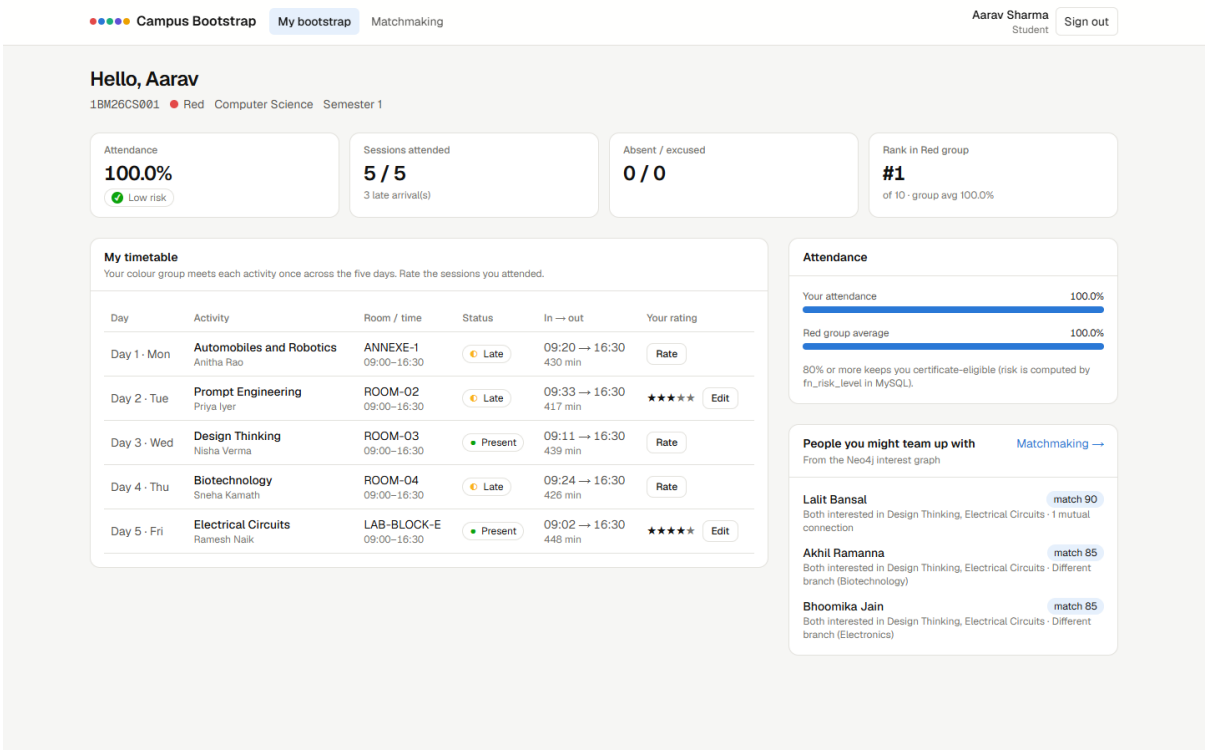


Fig 9.11: Student: timetable, attendance and rating

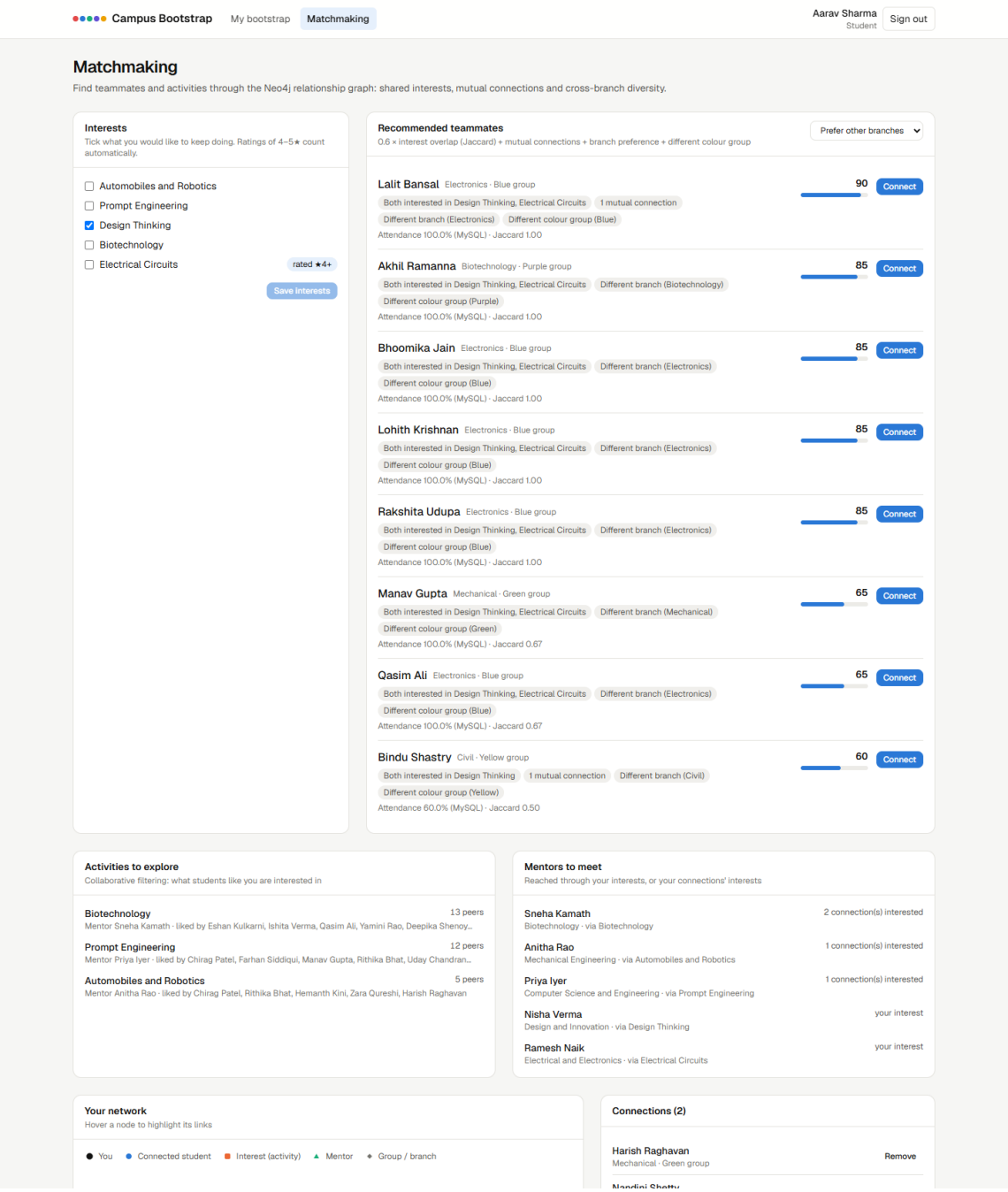


Fig 9.12: Student: matchmaking recommendations from Neo4j

9.4 Neo4j Integration

The graph has 76 nodes (51 Student, 10 Mentor, 5 Activity, 5 BootstrapGroup and 5 Branch). Each node carries its MySQL primary key, so the graph can be rebuilt from MySQL at any time. Table 9.2 lists the relationships.

Table 9.2: Graph relationships

Relationship	Count	Source
(Student)-[:BELONGS_TO]->(BootstrapGroup)	51	Student.g_id
(Student)-[:STUDIES]->(Branch)	51	Student.branch
(Activity)-[:MENTORED_BY]->(Mentor)	5	Activity.m_id
(Student)-[:PARTICIPATED_IN {status}]->(Activity)	255	Participates
(Student)-[:RATED {rating}]->(Activity)	76	Feedback
(Student)-[:INTERESTED_IN {source}]->(Activity)	101	A 4–5 star rating, or the student's own choice
(Student)-[:CONNECTED_TO]-(Student)	67	Students connecting in the app (stored only in Neo4j)

Teammate recommendations rank other students by the score below, and every suggestion shows the reasons behind it. The branch preference can be set to prefer other branches (the default, for cross-disciplinary teams), prefer the same branch, or neither.

$$score = 0.6 \times Jaccard(shared\ interests) + 0.15 \times min(mutual\ friends,\ 3) / 3 + 0.15 \times branch\ preference + 0.1 \times [different\ colour\ group]$$

Activity suggestions use collaborative filtering (activities that students with similar interests or connections like). Mentor discovery follows variable-length paths through interests and connections, and "how are two students connected?" uses a shortest-path query of up to six hops, which would need a recursive query in SQL.

Neo4j Browser queries

The screenshots below were taken in Neo4j Browser after the graph was synchronised from MySQL. Each one shows the Cypher query, the graph it returns, and the node and relationship counts.

The first query checks the import. Every node type matches its MySQL table: 51 Student, 10 Mentor, 5 Activity, 5 BootstrapGroup and 5 Branch nodes, 76 in total.

```
MATCH (n)
RETURN labels(n) AS NodeType, count(n) AS Total;
```

Table | Raw

NodeType	Total
["BootstrapGroup"]	5
["Mentor"]	10
["Activity"]	5
["Student"]	51
["Branch"]	5

Started streaming 5 records after 84 ms and completed after 86 ms.

Fig 9.13: Node count by label

The relationship counts also match MySQL: 255 PARTICIPATED_IN (one per Participates row), 76 RATED (one per Feedback row), and 51 each of BELONGS_TO and STUDIES.

```
MATCH ()-[r]->()
RETURN type(r) AS RelationshipType, count(r) AS Total;
```

Table | Raw

RelationshipType	Total
"MENTORED_BY"	5
"INTERESTED_IN"	101
"BELONGS_TO"	51
"STUDIES"	51
"PARTICIPATED_IN"	255
"RATED"	76
"CONNECTED_TO"	67

Started streaming 7 records after 47 ms and completed after 50 ms.

Fig 9.14: Relationship count by type

Each student belongs to exactly one colour group, so the graph splits into five separate clusters, one per group.

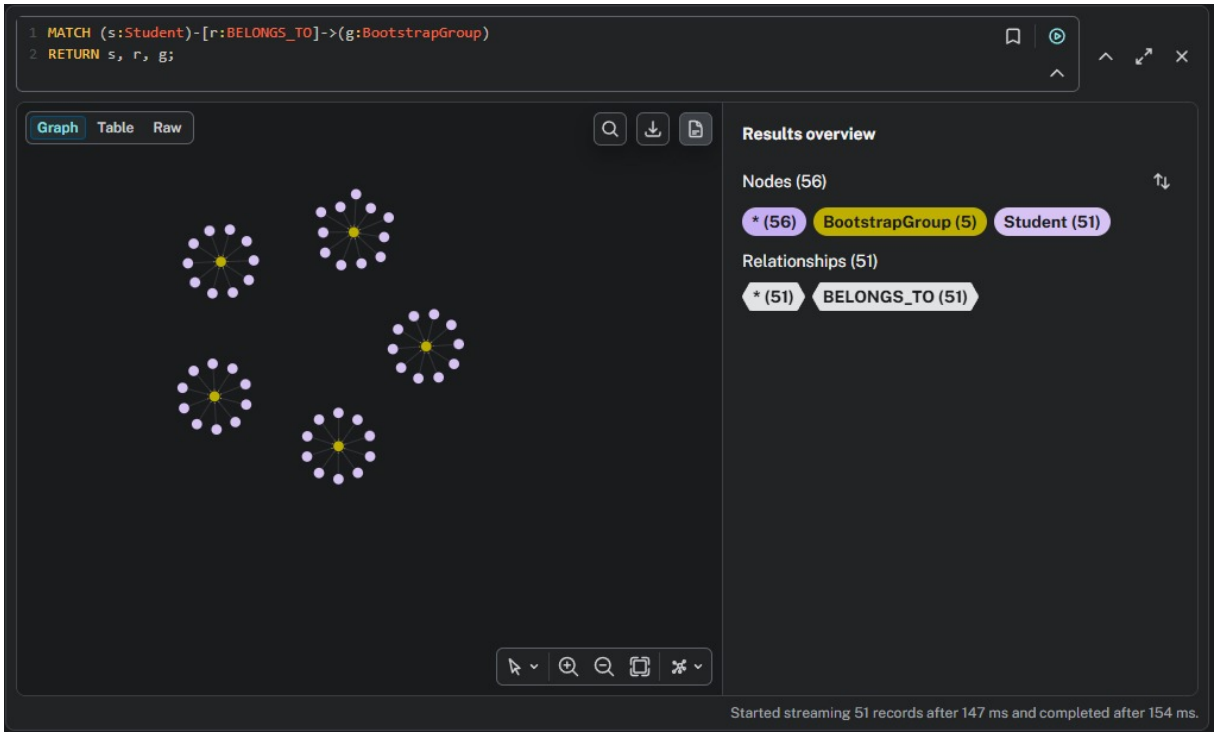


Fig 9.15: Students and their colour groups (BELONGS_TO)

Branch is stored as a column in MySQL but as its own node in Neo4j. This lets the matchmaking queries check in one step whether two students are from different branches.

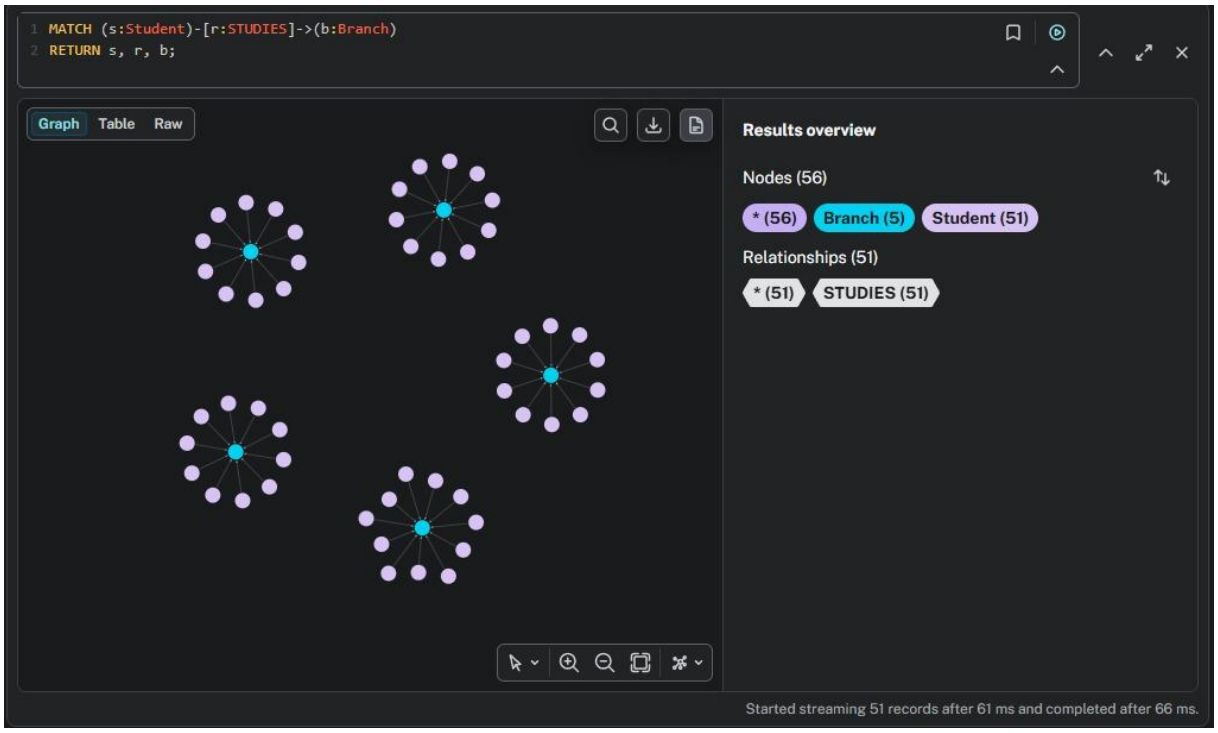


Fig 9.16: Students and their branches (STUDIES)

Every student is linked to all five activities, because every group visits every activity once. The attendance status is stored as a property on each relationship.

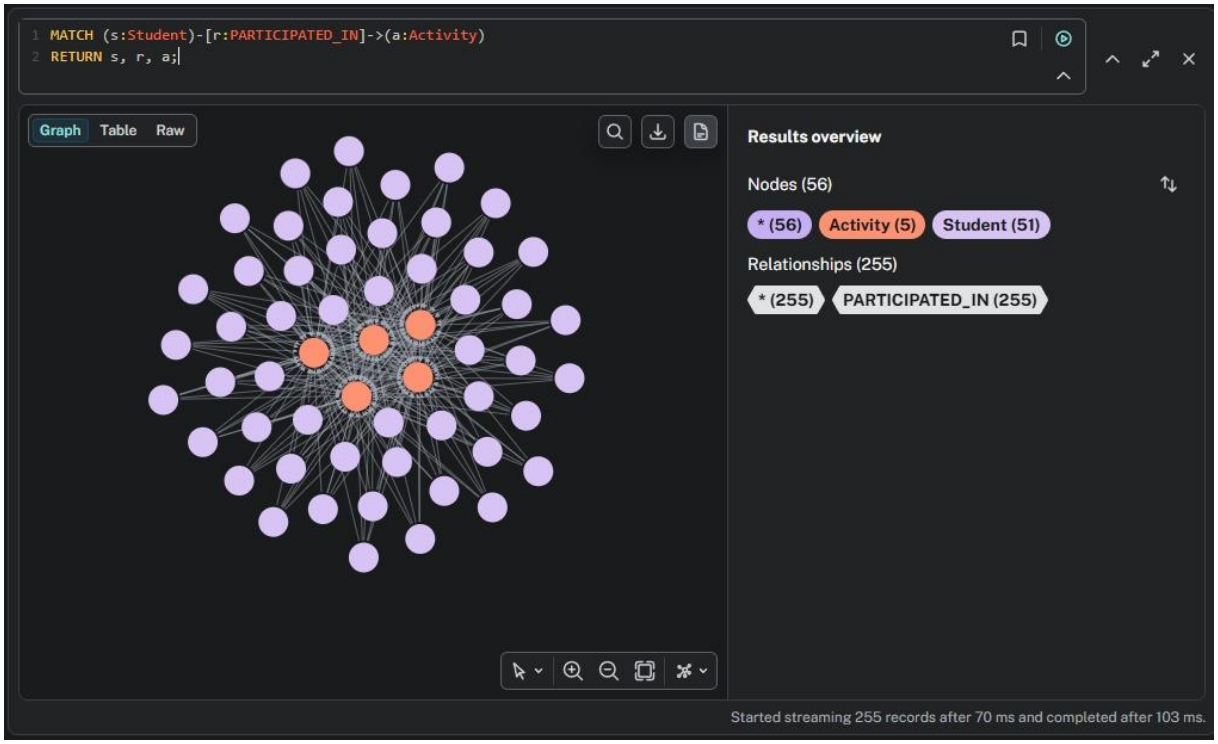


Fig 9.17: Participation of every student in the five activities (PARTICIPATED_IN)

Interests come from two sources: a rating of 4 or 5 stars in MySQL, or the student ticking the activity on the Matchmaking page. Students who share an interest are candidates for teammate recommendations.

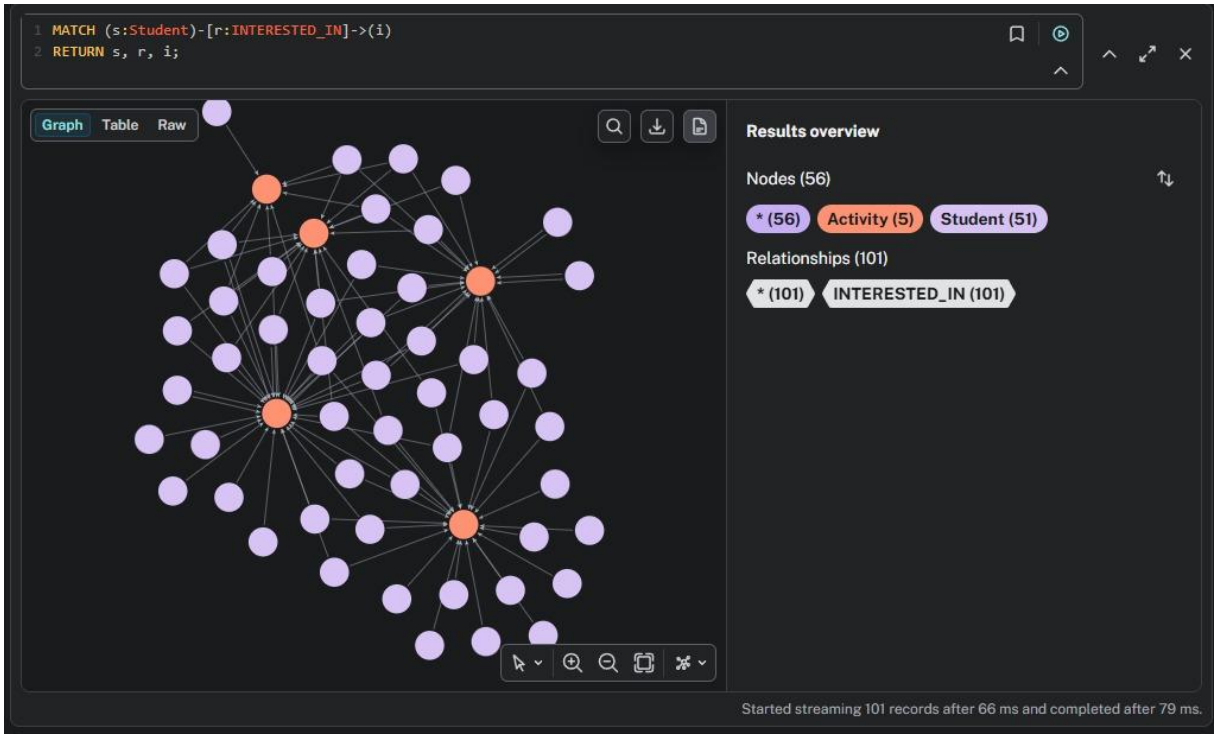


Fig 9.18: Student interests (INTERESTED_IN)

Connections are made when a student clicks Connect in the app and are stored only in Neo4j. They are used for mutual-friend scoring and for finding the shortest path between two students.

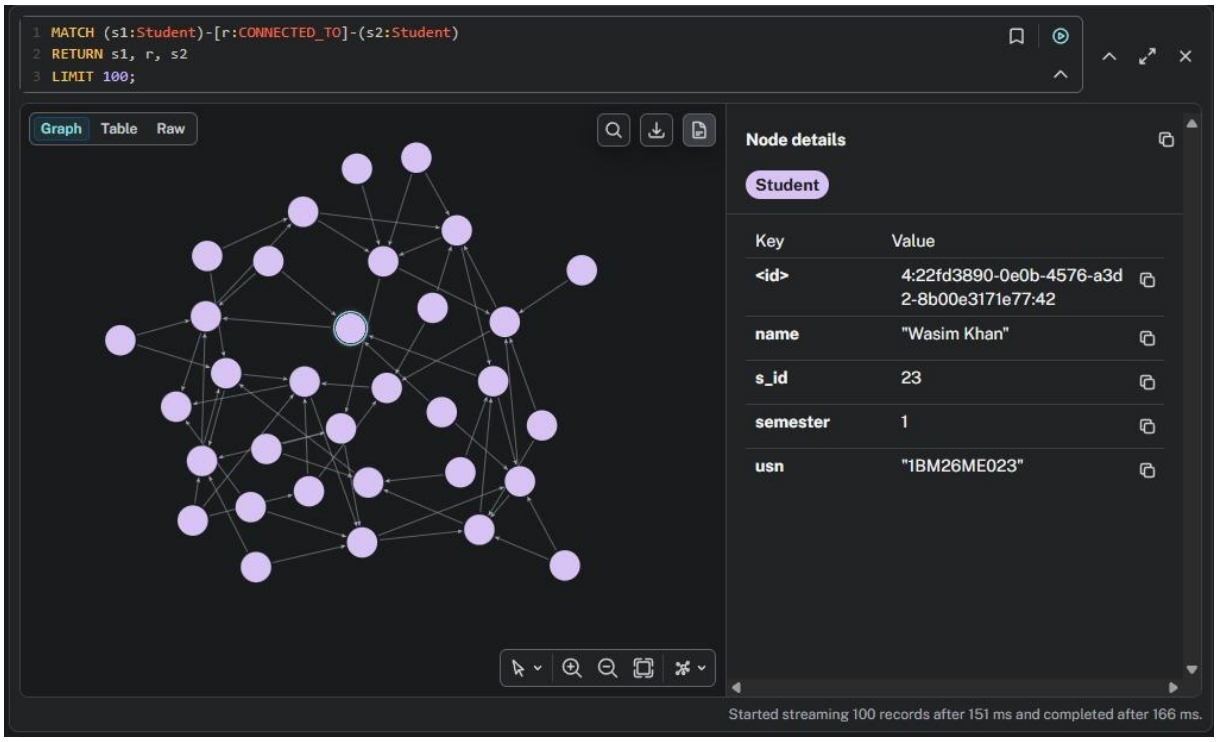


Fig 9.19: Peer connections between students (CONNECTED_TO)

Putting the group, branch and interest relationships together shows the information the teammate score uses: shared interests, plus a bonus for a different branch or a different colour group.

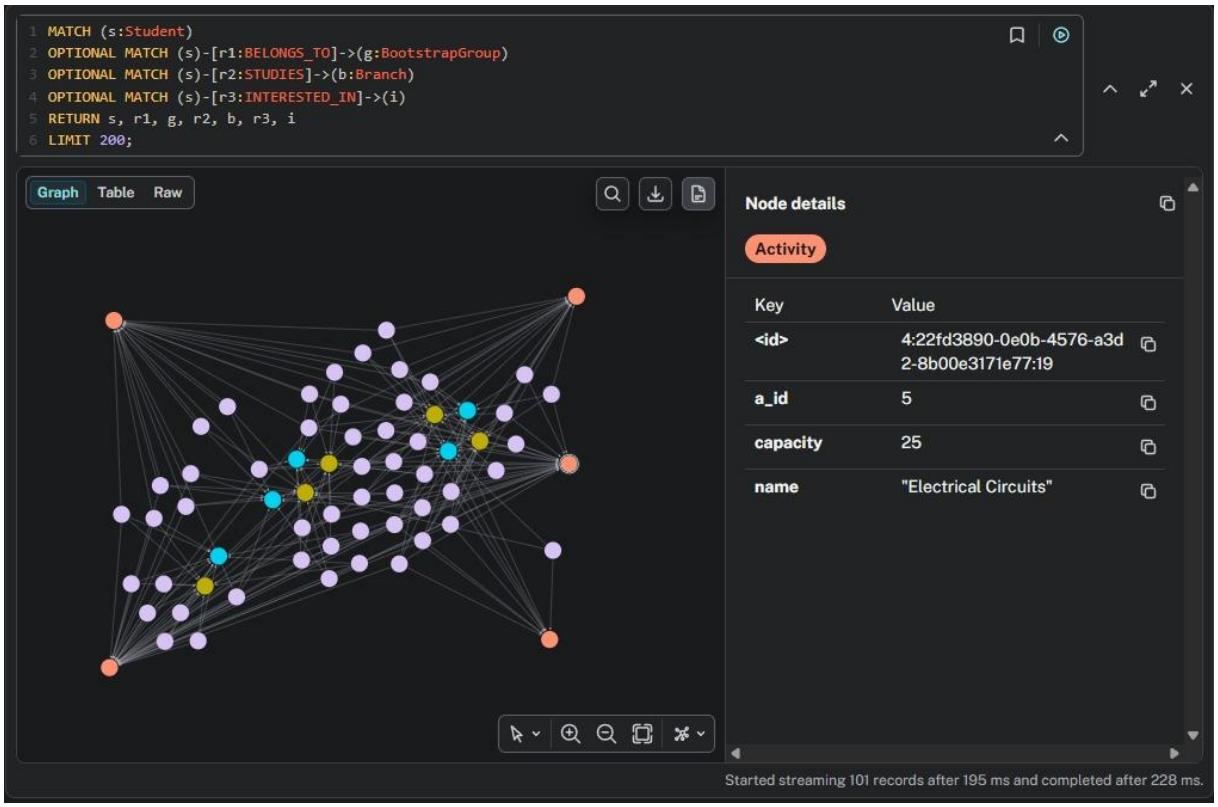


Fig 9.20: Students with their group, branch and interests

Finally, a sample of 200 relationships of all types shows how students, activities, groups, branches and mentors are connected in one graph.

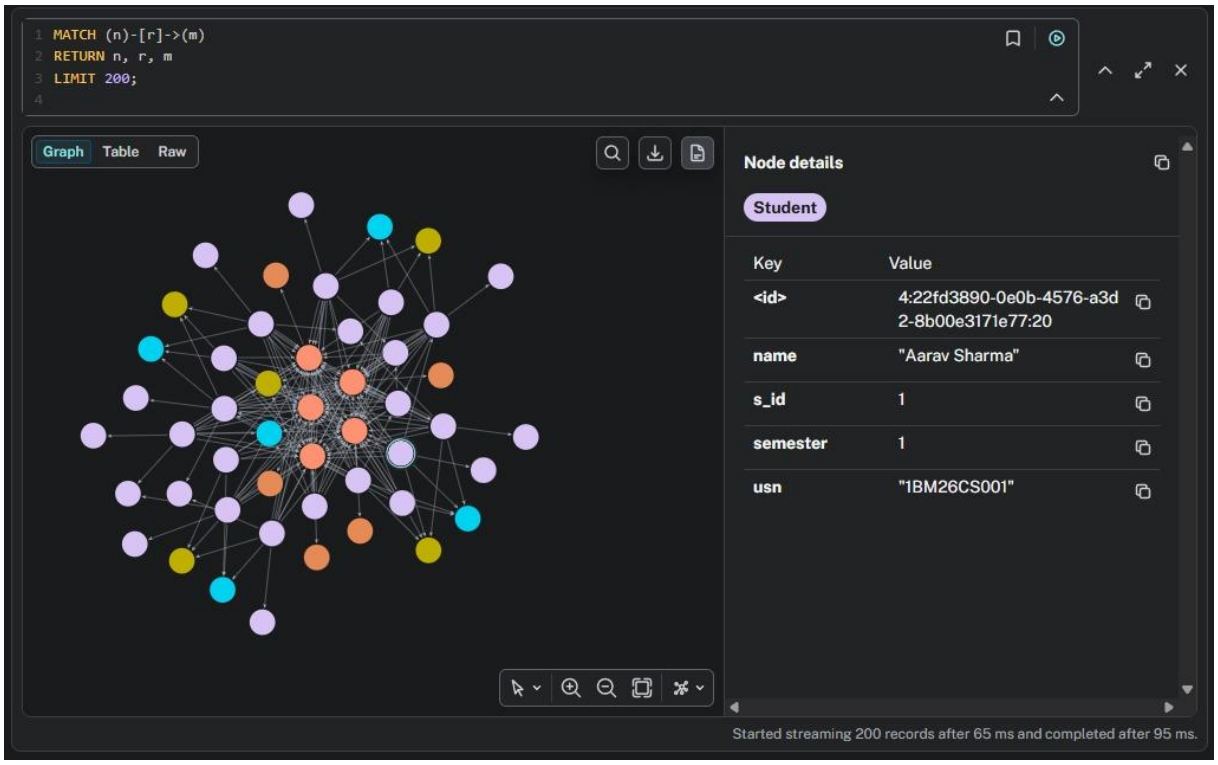


Fig 9.21: A sample of the whole graph

9.5 Security

- Passwords are stored only as bcrypt hashes, and a CHECK constraint refuses anything else.
- Sessions are signed JWTs in an httpOnly cookie that expires after 8 hours; five failed logins lock an account for 15 minutes.
- Every endpoint checks the user's role, students are restricted to their own records, and mentors to their own activities.
- All SQL and Cypher values are passed as parameters, which prevents injection.
- Secrets are kept only in a git-ignored environment file, and a least-privilege MySQL account (SELECT, INSERT, UPDATE, DELETE and EXECUTE only) is provided for deployment.

9.6 Testing

The system is tested with Vitest at three levels. Unit tests cover validation, authentication and scoring. Database-integration tests run against MySQL and Neo4j and check the schema, every trigger, the transactions and the graph sync; all 69 unit and integration tests pass. End-to-end tests call the HTTP

API of the running application, including the path from a 5-star rating in MySQL to a new recommendation from Neo4j.

REFERENCES

[1] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Pearson, 2016.

[2] A. Silberschatz, H. F. Korth and S. Sudarshan, Database System Concepts, 7th ed. McGraw-Hill, 2019.

[3] Oracle Corporation, “MySQL 8.0 Reference Manual.” https://dev.mysql.com/doc/refman/8.0/en/

[4] Neo4j, Inc., “Cypher Manual.” https://neo4j.com/docs/cypher-manual/current/

[5] Neo4j, Inc., “Neo4j JavaScript Driver Manual.” https://neo4j.com/docs/javascript-manual/current/

[6] Vercel, “Next.js Documentation.” https://nextjs.org/docs

[7] “MySQL2: MySQL client for Node.js.” https://sidorares.github.io/node-mysql2/docs

GitHub Link: https://github.com/thisisselva18/bootstrap-db/